# Resource-Constrained Annotation Workflows for Paediatric Histopathology Reports using LLMs

Avish Vijayaraghavan[abcd*#], Jaskaran Singh Kawatra[d], Sebin Sabu[d], Jonny Sheldon[d], Will Poulett[c], Shiren Patel[d], Neil J. Sebire[d]
Jonny Pearson[c△], Dan Schofield[cd△], Pavi Rajendran[d△], Jonathan Hope[c△]

[a]Division of Systems Medicine, Department of Metabolism, Digestion and Reproduction, Imperial College London

[b]Department of Computing, Imperial College London

[c]Data Science Team, NHS Transformation Directorate

[d]GOSH DRIVE, Great Ormond Street Hospital

[*]Corresponding author: Avish Vijayaraghavan; avish.vijayaraghavan17@imperial.ac.uk

[#]Work done during a internship at NHS Transformation Directorate / GOSH DRIVE.

[△]Jointly supervised this work.

**Abstract**

Extracting structured information from unlabelled medical reports remains a significant challenge in healthcare, particularly for histopathology where reports contain domain-specific terminology with highly-variable phrasing, even within one disease area. This variability makes the manual information extraction process inefficient, often taking multiple iterations to deliver satisfactory results, and necessitating clinical experts who already have scarce time for rote tasks like these. The variability challenge is even more pronounced in the paediatric setting where datasets are smaller and patient presentations can differ significantly, making it harder to extract general patterns. The advent of flexible zero-shot and few-shot language models has presented an opportunity to tackle this problem. In this work, we propose a resource-efficient annotation workflow for structured information extraction that combines smaller language models (<5B parameters) with a minimal number of data scientist and clinician touchpoints. Our system is designed specifically for constrained clinical environments, operating on CPU infrastructure with sensitive data that cannot leave local servers. Experiments on an annotated sample of paediatric histopathology reports for renal biopsies from Great Ormond Street Hospital demonstrate efficient (<1d to process 2000 reports on an NHS laptop), flexible (adaptive to different annotation schemas) and accurate (>80% accuracy) structured information extraction, through a combination of carefully-crafted entity guidelines and few-shot samples. An implementation is available as an open-source Python codebase at www.github.com/gosh-dre/nlp_renal_biopsy.

## 1 Introduction

Histopathology, the microscopic examination of tissue samples, is fundamental to disease diagnosis and treatment planning. A simplified process is shown in Figure 1: a biopsy is

performed and stained, pathologists examine images of tissue slides and document their findings in detailed reports, which serve as records for clinical decision-making. However, these reports often contain complex, unstructured text that makes systematic analysis challenging. Automating the annotation of histopathology reports could enhance patient care in multiple ways: standardising the documentation process, reducing clinician workload, enabling scalable insights for research, and supporting quality audits. Our study aims to advance this vision by framing the annotation task as a question-answering (QA) task, and using large language models (LLMs) to perform QA over unlabelled histopathology reports. This approach not only supports downstream machine learning tasks but also lays crucial groundwork for vision-language models (VLMs) in paediatric settings (i.e., small highly-variable datasets), where high-quality report annotations are needed for robust training signals.

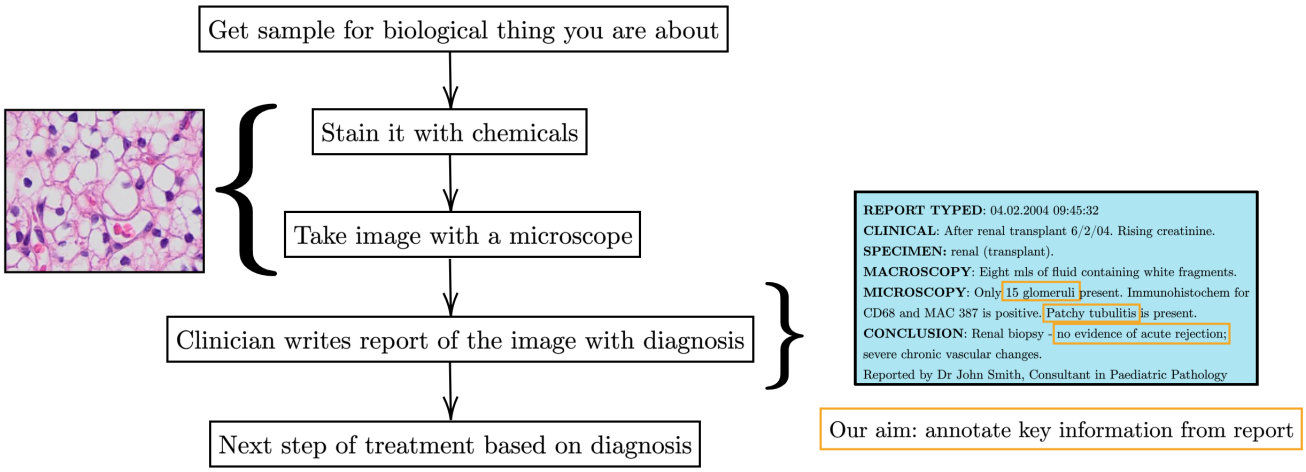



Figure 1: The general histopathology workflow. Our focus is on extracting key text spans from the clinician's report that are relevant for diagnosis.

A fundamental challenge in working with histopathology reports lies in the nature of medical free text. These reports contain domain-specific terminology, abbreviations, informal language, and occasionally noisy or misspelled text. A comprehensive study by the King's Fund Trust [1], analysing over one million clinical documents, confirmed that a significant portion of clinical communication occurs in free text format, highlighting the importance of addressing this domain-specific language variability. Additionally, these reports often include ambiguities and shorthand notations that, while intuitive for human clinicians, pose significant challenges for traditional natural language processing (NLP) models.

Our proposed approach is a question answering (QA) pipeline that differs from other NLP approaches like ontology lookups and named entity recognition (NER). While dictionary- or ontology-based methods (such as SNOMED-CT keyword lookup) would suffice for simple diagnosis prediction, our goal requires capturing detailed symptom information for accurate report summarisation. NER, while effective for straightforward tagging tasks, struggles with the variability, longer spans, and implicit entity relations within medical text. Additionally, histopathology reports often contain meta-annotations like negations (e.g. "there is no evidence of rejection") and uncertainties (e.g. "there are features suggestive of chronic tubulointerstitial nephritis"). Standard medical NER models like medspaCy [2] prove too limited, and while more advanced models like MedCAT [3] can handle meta-annotations, they do not capture the variability of entity descriptions. Instead, we turn to QA (which

subsumes NER) using LLMs, which have shown good performance on language tasks that do not require precise localisation or categorisation of entities. LLMs have greater task flexibility because their inputs (called "prompts") can be customised in natural language and so can be easily modified to a specific biological domain. Their flexibility also makes them easy to integrate into existing clinical workflows, aligning with our overall aims of automated annotation [4].

This study details our development of automated annotation systems at Great Ormond Street Hospital (GOSH), a leading tertiary paediatric centre in London. This setting presents several constraints: the sensitive nature of paediatric health data that cannot leave local servers, limited computational infrastructure restricted to CPU processing, scarce clinician time for annotation tasks, and unlabelled medical report datasets. These challenges are compounded by both the inherent complexity of paediatric medicine, where patient presentations vary considerably, and referral pathways into GOSH, which can bring patients from all over the country introducing additional unknown confounders. This is important as there are only a few language models trained on or used for paediatric text [5, 6, 7, 8], despite (or, perhaps, due to challenges of) the known high variability in paediatric clinical reporting. Given these constraints and challenges, we focus on large language models on the order of 100M-5B parameters[1], specifically investigating quantised generative models with few-shot prompting. These models offer a pragmatic balance between computational efficiency and the ability to capture complex medical narratives, alongside easier interfacing with non-computational practitioners.

Histopathology reports vary considerably depending on the chosen disease and biopsy. To navigate this broad area of medicine, we demonstrate our proof-of-concept annotation workflow on a relatively constrained and well-understood "test case" within histopathology: renal biopsies. Renal biopsies involve a narrower range of pathological triggers and diagnostic outcomes (Figure 2), making them ideal for testing the robustness of our proposed machine learning pipeline alongside human evaluation. Since histopathology as a whole is a well-established diagnostic modality, renal biopsy reports offer a safe yet informative starting point. Success here could pave the way for extending these models to more complex and less standardised pathology subfields.

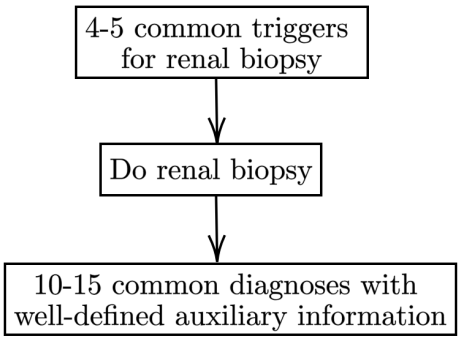



Figure 2: Complexity of renal biopsy pathway.

Our contributions are as follows:

1. **Human-Centred Annotation Workflow.** We design a resource-constrained annotation workflow for renal biopsies using LLMs with three critical clinician touchpoints.

[1]Models of this size have also been referred to as "small language models" in AI literature. We have chosen to stick to the phrase "large language models" for simplicity.

2. **Python Codebase.** We implement the annotation workflow as a Python codebase with LLM-based entity annotation (with options for ollama and llama-cpp-python as inferencing servers), LLM-based disagreement modelling, an interactive Streamlit app for annotation, and an interactive Streamlit app for comparing predictions from different models.

3. **Experiments.** We perform experiments comparing different structured extraction methods, our annotation workflow with different LLMs, and the effect of prompt engineering, specifically, the use of entity guidelines and few-shot samples in the input prompt, and tweaking the evaluator LLM prompt.

# 2 Related Work

## 2.1 LLMs for Biomedical NER into QA

The emergence of word embeddings marked a shift towards deep learning-based biomedical NER. scispaCy [9], for instance, integrated UMLS linkages and offered pre-trained embeddings tailored to biomedical texts. Biomedical takes on the transformer architecture BERT (e.g., BioBERT [10], ClinicalBERT [11]), significantly advanced the field by capturing contextual information and long-range dependencies. MedCAT [3] built upon this by incorporating spelling correction, contextual disambiguation (e.g., differentiating "hr" as "hour" or "heart rate"), and flexible entity linking to UMLS and SNOMED-CT, improving standardisation potential. These BERT-based models outperformed earlier systems on benchmark datasets but often require fine-tuning on large annotated corpora, which limits their applicability in resource-constrained environments.

Recent NER models have focused on adaptability to unseen entities at test time, often using BERT backbones within their overall architectures. GLiNER [12] approaches NER as a compatibility task between entity type prompts and text spans, using two-layer FFNNs to compute embeddings for both components. The model uses a cross product of these embeddings wrapped in a sigmoid to calculate the probability that a span $(i, j)$ is of entity type $t$. Its flexibility here supports user-defined labels and handling flat and fully-nested NER. calculates match probabilities through cross-product scoring with sigmoid activation, supporting user-defined labels and handling flat and fully-nested NER. However, GLiNER cannot process discontinuous spans or partially-nested NER, limiting its application in medical reports where more complex expressions of symptoms are possible.

Question answering (QA) frames information extraction as a reading comprehension task, where models must locate and extract relevant information from input text to answer specific questions. While NER outputs are limited to entity-type pairs, QA can generate free-form answers that combine multiple pieces of information, reason across paragraphs, and handle implicit relationships between concepts. There is some form of information synthesis that happens in QA unlike NER. Alongside BERT-backbone QA models, the field has also explored using generative LLMs (i.e. unidirectional architectures building off GPT models, unlike the bidirectional BERT-based architectures that predict masked tokens) for NER by reformulating NER as a text generation task [13, 14]. This approach includes input sentences in pre-defined prompt templates, treating entity annotation as a fill-in-the-blank (i.e. text generation) problem as opposed to traditional sequence labelling. More recent methods like NuNER [15] and NuExtract [16] have extended these ideas by combining BERT-based models with data generated from GPT-style models and fine-tuning generative models for structured output extraction, respectively. These prompt-based NER models exist in the hazy

space between NER and QA, or to be even more precise, the hazy space between extractive QA (where answers must be in the text) and open generative QA (where answers can be generated based on the text). QA is useful because it can capture document-level context but is not without flaws - it is more computationally expensive, and, when using inference techniques, prone to making things up [17].

## 2.2 LLMs for Pathology Reports

Large language models (LLMs) have been applied to a variety of pathology reports, with different focuses: domain-specific tokenisation, active learning, data augmentation, big datasets, and localised models. We note that the majority of these papers have much larger datasets than our dataset of 2462 reports, bar Zeng et al's paper [18], and our focus is mostly on human-integrated annotation workflows to maximise reliability and interpretability.

PathologyBERT [19] introduced a custom tokeniser to address limitations of the WordPiece tokeniser, which can break down medical terms in ways that lose semantic meaning (e.g., "carcinoma" into ["car", "cin", "oma"]). Using 340,492 unstructured histopathology reports from 67,136 patients at Emory University Hospital between 1981-2021 alongside with labelled set of 6,681 reports from 3,155 patients, they demonstrated greater coverage of pathology-specific terminology compared to previous clinical language models like BlueBERT [20] and ClinicalBERT [11]. Their reports followed a semi-structured template like ours: history, diagnosis, gross description, microscopy examination, comments. However, unlike our data, their reports are much shorter with an average report length of 42 +- 26 tokens.

Mu et al. [21] generated diagnostically relevant embeddings from 11,000 bone marrow pathology synopses using a BERT model enhanced with active learning. The synopses followed the structure of: report (tissue morphology), diagnosis (stuff from pathology specimen, ancillary testing, clinical history). The active learning approach minimised manual annotation efforts while achieving strong performance, with only 350 annotated synopses required to reach an F1 score plateau, outperforming random sampling strategies. Zeng and colleagues [18] trained a BERT-based NER model on 1,438 annotated US pathology reports and tested it on a separate dataset of 55 reports from the UAE. They found that data augmentation strategies, such as mention replacement, synonym replacement, label-wise token replacement, and segment shuffling, led to more accurate entity recognition for cancer grades, subtypes, and lesion positions compared to LSTM models.

Lu et al. [22] analysed two key datasets, (1) a dataset of 93,039 reports with CPT codes, and (2) a bigger general dataset of 749,136 reports, examining various classification tasks. Their analysis revealed differences in entity complexity across categories, with position-related entities showing high variability while cancer grades had more constrained vocabularies. They also found that medical domain pretraining did not consistently improve perfromance. Bumgardner and colleagues [23] explored the use of local large language models (LLMs) for extracting structured information from surgical pathology reports. The task was complicated by the presence of multiple tissue specimens with varying levels of malignancies within a single case. Their reports followed a rough schema of gross descriptions (characteristics of tissue specimens), final diagnosis (diagnosis based on microscopy, lab results and clinical notes), and condition codes. Using 150,000 reports they evaluated several models, finding a quantised LLaMA 13B model outperformed others, including the Path-LLaMA variant which was instruction-tuned for pathology tasks.

## 2.3 LLMs for Human-Integrated Annotation Workflows

A recent review paper [24] has split this nascent field up into three categories: annotation generation, annotation assessment, and annotation utilisation. The paper has further categorisation of eight tasks, of which our work loosely falls into four: categorising raw data with labels, adding intermediate labels for contextual depth, confidence scores for assessing annotation reliability, and tailoring outputs to user needs. The review takes a bigger focus on knowledge distillation - our work differs emphasising asynchronous human-integrated annotation workflows using only smaller models in medical settings settings. Here, we discuss some relevant example papers and how they have guided our final annotation workflow in Figure 3.

One common approach is to use iterative strategies for annotation. FreeAL demonstrates an iterative knowledge distillation framework where GPT-3.5 Turbo generates weak labels for unsupervised data, while RoBERTa-Base filters for high-quality annotations [25]. This recursive process addresses LLM confirmation bias through iterative refinement, where the smaller model improves the annotation pool to enhance the LLM's subsequent performance. In the biomedical domain, Munnangi et al. [26] developed an entity definition-specific approach combining UMLS concept definitions with entity linking . Their two-stage approach for NER allowed models to use definitions to revise predictions through entity addition, removal, or type reassignment, while using scispaCy for mapping biomedical concepts to a human-curated set of relevant UMLS entries. Naraki et al. [27] use label mixing to deal with entity class imbalance, prompt templates with strict entity definitions and missing entity rules, and a downstream BERT model to evaluate annotations. This work is the most similar reference point to our work; however, we have fairly balanced entity occurrence, take a broader focus on the workflow, and choose to incorporate expert knowledge specific to medical reports.

A broad philosophy for ensuring reliable annotations without humans is some form of ensembling. The T-SAS framework [28] demonstrated effective annotation using unlabelled test data through stochastic answer generation, employing Monte-Carlo dropout with majority voting and ensemble filtering to mitigate noise. A blog post by Warmerdam [29] recommends disagreement modelling as a simpler alternative to active learning for refining annotations, using error analysis in a feedback loop to produce better inputs for the models (e.g. refining prompts). Stochastic methods with filtering is a useful strategy which we have not been able to use due to compute limitations but served as equal inspiration to using disagreement modelling in our eventual workflow (Figure 3).

An important consideration for us is finding the critical parts of the annotation workflow where expert knowledge is needed. Explosion (an industry NLP lab) has emphasised finding the balance between humans and LLMs by considering their respective strengths and limitations. Humans are great at contextual understanding and resolving ambiguity but worse than machines at tasks requiring extensive memory and consistent repetition [30]. They note we have strong limitations on how much information we can understand at one time and suggest focussing on the minimum required information for a human when requesting feedback. A recent paper by Gao et al. [31] proposed CliniCoCo - a framework aligning AI models with human-in-the-loop feedback for better clinical coding workflows. CliniCoCo has a range of human touchpoints: selective annotation during preprocessing, active learning during training, and customisable explainable AI workflows at the final clinical decision-making stage. The findings from Explosion and CliniCoCo were inspiration for minimising and optimising the expert touchpoints in our final workflow.

# 3 Methods

Our overall workflow for annotating reports and performing question answering consists of multiple steps in a feedback loop with three key clinician touchpoints: (1) preprocessing the raw data into an appropriate format and performing model-appropriate EDA, (2) creation of the annotation guidelines and resulting prompts, and (3) a modelling workflow where models and prompts are optimised, before (4) the final automated annotation workflow. The task was originally framed as named entity recognition (NER) but later converted to a question answering (QA) task - we explain the results that led to this decision in the Discussion.

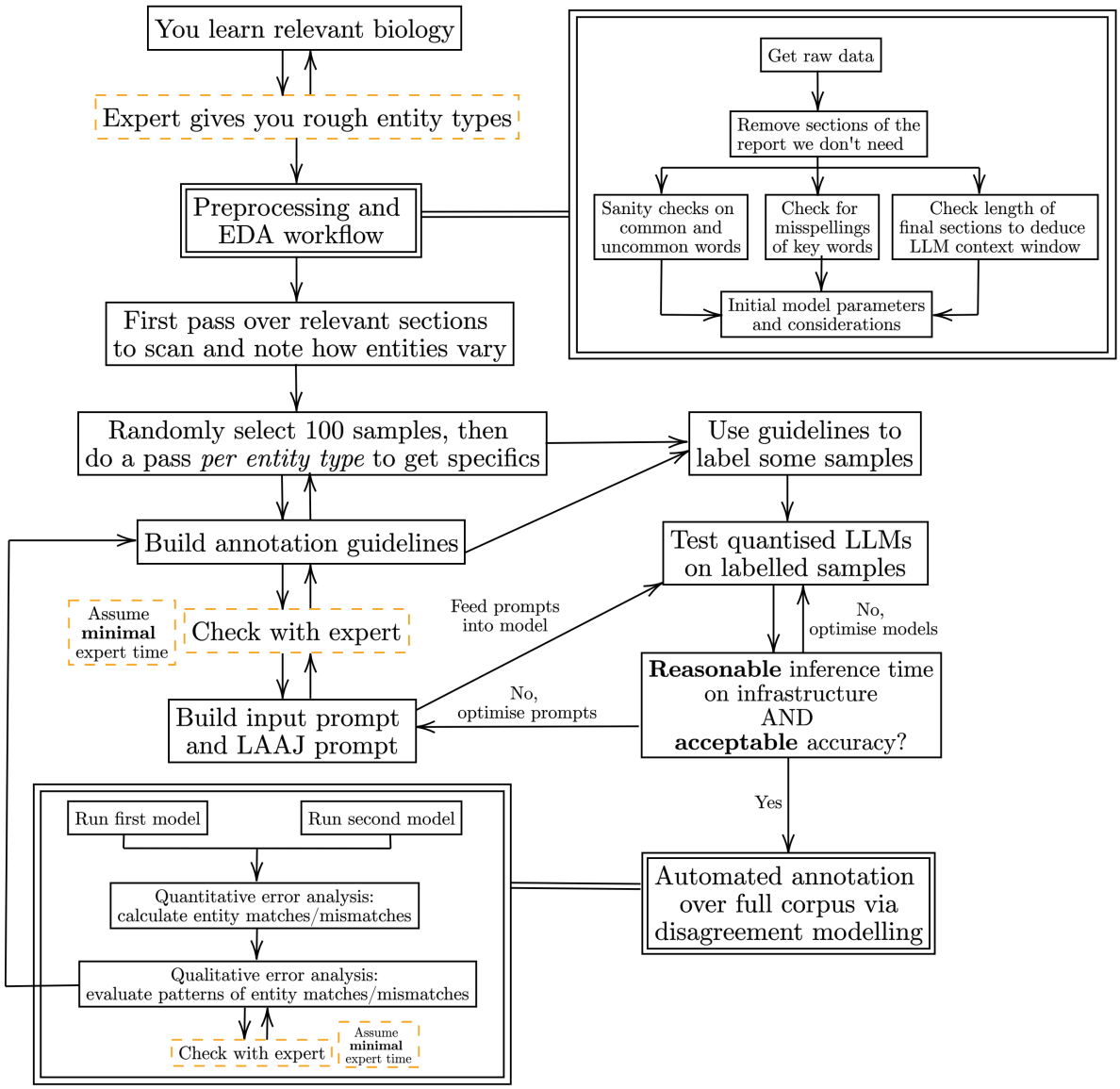



Figure 3: Annotation workflow for medical reports which three expert touchpoints highlighted with orange dotted lines. The second and third touchpoints have potential to be indefinitely long and so we have specified the assumption of minimal expert time.

## 3.1 Constraints and Workflow Requirements

Our work operates under several practical constraints that shape our methodology. First, clinician time is highly limited, with consultant pathologists only available for approximately

one hour per month: 30 minutes for meeting and 30 cumulative additional minutes for annotating reports in their spare time. This necessitated efficient use of their expertise, with a focus on understanding relevant biology and figuring out guidelines for edge cases. Second, the sensitive nature of the paediatric histopathology data prevented the use of external servers, including standard large language model (LLM) APIs, requiring us to adopt local LLMs for all computational tasks. Third, resource limitations influenced the choice of local infrastructure. Analysis was limited to CPU-based hardware, visualised in Figure 4: a GOSH laptop (with approved credentials) with 16GB RAM and GOSH's computational infrastructure called GOSH Research Intelligence Development (GRID). GRID consists of two servers: a less capable development ("dev") server with 4 cores and 16GB RAM which requires deidentified data and has access to the internet, and a more capable staging server with 12 CPU cores and 100GB RAM that does *not* have access to the internet. The laptop and dev server are suitable for prototyping ideas on subsets of data and the staging server is required to run models over the full dataset. The staging server is used for training and inference purposes. The staging server is a pre-production environment and uses container images developed on the dev server.

The guiding question for our workflow is "how can LLMs help clinicians label data faster?" From this, and given our constraints above, our modelling requirements are: reasonable inference time on GOSH infrastructure (this can loosely be defined as two days as per the threshold for an "extended run time" for GOSH laptops [32]), acceptable accuracy, and minimal impact to existing clinician workflows, all specified in Figure 3.

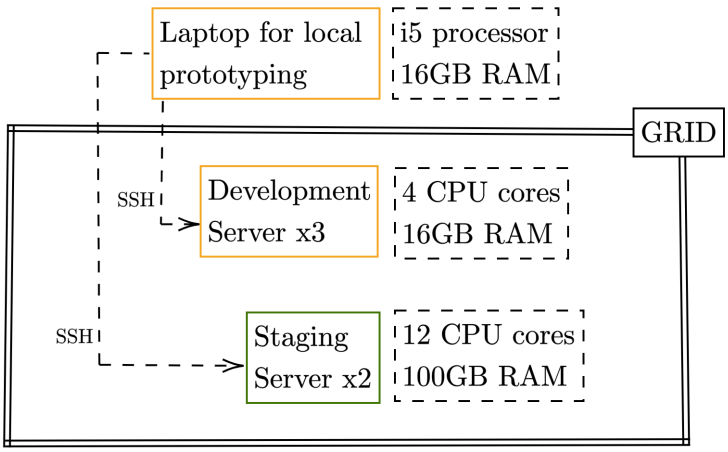



Figure 4: Diagram showing GOSH's compute.

## 3.2 Report Dataset, Preprocessing, and EDA

**Histopathology Reports for Paediatric Renal Biopsies**. We have 2462 reports from 1581 patients and used a random sample of 100 reports to determine our initial preprocessing workflow and guidelines, as per Figure 4. Renal biopsy pathology interpretation focuses on two key structures within the kidney's million nephrons: glomeruli, where fluid passes into Bowman's capsule while retaining larger molecules like proteins and white blood cells (WBCs), and the surrounding kidney tissue. The pathological findings differ between native and transplant biopsies. Native biopsies typically show glomerular abnormalities, while transplant biopsies focus on signs of rejection like tubulitis and infiltrate (which is the name for combinations of WBCs coming into a certain area of the body) in the tubulointerstitial

space. Key pathological changes include sclerosis (affecting the glomerulus interior) and fibrosis (affecting surrounding kidney tissue). Our entity schema builds off the Banff classification [33], a renal transplant staging system, to accommodate both native and transplant biopsies. This adaptation uses fewer, more general entities while maintaining detailed classification of glomerular patterns and reference to other blood vessel abnormalities. These seven initial entities with their data types were provided by the clinician (Figure 1) - we describe development of this schema in Section 3.3 and show the final version in Section 4.2.

Table 1: Initial entities and their data types from the consultant clinician.

| **Entity Name** | **Data Type** |
|---|---|
| cortex | Binary |
| medulla | Binary |
| number of glomeruli | Numerical |
| number of sclerosed glomeruli | Numerical |
| % chronic change/fibrosis | Numerical |
| glomeruli normal or abnormal | Binary |
| final diagnosis | Categorical/Code |

**Preprocessing**. The preprocessing pipeline was designed to structure unlabelled histopathology reports for effective question answering with LLMs. Reports consist of different sections, mostly following the example in Figure 5. We collected a random sample of 100 reports (∼4% of the overall dataset) to use as our dataset for developing the first version of our pipeline. Raw text from the reports was converted into a structured dictionary format using regular expressions, with each (key, value) pair corresponding to a different section heading and its content. We used a Levenshtein distance of two to deal with any header misspellings (we found one that had "CFONCLUSION" instead of "CONCLUSION") and handled one edge case ("ELECTRON MICROSCOPY" interfering with the standard "MICROSCOPY" header) by hardcoding it in. For our overarching goal of creating a strong signal for the report's histopathology image, we only needed the "MICROSCOPY" and "CONCLUSION" sections and so could remove the others, which has the added benefit of creating a smaller prompt for quicker inference. The "CONCLUSION" section is followed by a line stating which clinician has completed the report. Discussion with the clinician confirmed that we should not treat this as a confounder for now, and so we removed it.

**Exploratory Data Analysis (EDA)**. We use `scikit-learn`'s `count` and `tf-idf` functionalities to find common and uncommon words to make sure the common words overlap sufficiently with our chosen entities. We check for misspellings in our key words using the Levenshtein distance. Finally, we look at the lengths of the chosen sections to determine the minimum LLM context window needed. Although common in medical free text, our reports had no unique abbreviations or non-standard characters, simplifying this process.

## 3.3 Entity Guidelines and Prompt Creation

Next, we detail the workflow to develop entity guidelines and subsequently create the prompts for our LLM. A consultant clinician in paediatric pathology provided the initial set of en-

**REPORT TYPED**: 04.02.2004 09:45:32

**CLINICAL**: After renal transplant 6/2/04. Rising creatinine.

**SPECIMEN**: renal.

**MACROSCOPY**: Eight mls of fluid containing white fragments.

**MICROSCOPY**: Only 15 glomeruli present. Immunohistochem for CD68 and MAC 387 is positive. Patchy tubulitis is present.

**CONCLUSION**: Renal biopsy (transplant) – no evidence of acute rejection; severe chronic vascular changes.

Reported by Dr John Smith, Consultant in Paediatric Pathology.

Figure 5: Example Renal Biopsy Report

tities for us to refine. We conducted an initial review of approximately 4% of the reports (100/2462 samples) over a period of two weeks, identifying key entities, how they varied, and catalogued associated meta-annotations such as negation and uncertainty. We kept a bank of edge cases to refer back to later on during the prompt creation step. From this we determined our initial version of the entity guidelines schema and refined this after discussion with the clinician - the finished version can be viewed in the public repository and a simplified version with key things to make note of when analysing reports can be viewed in Table 4.

The entity guidelines schema - the choice of entities, their corresponding types, additional information on entity variability, and potential combinations of entities - should balance biomedical faithfulness, clinical utility, and simplicity. Our `guidelines.xlsx` file details the possible entity types: *boolean* (T/F), *categorical*, *numerical*, *string-simple* (a proper noun or short phrase), *string-complex* (longer strings, often used for descriptions or explanations). Note that schema complexity is a function of number of entities and entity type complexity. The simplest case is a minimal set of clinically-actionable boolean entity types - this is both easy to evaluate and turn into a subsequent decision - but unfortunately, may lose key information about the pathological process, and lose interpretability if reasoning is required. Some entity types were broken into more granular sets of entities. For example, our initial specification wanted the *number of sclerosed glomeruli* but we realised that it was crucial to have reference to the level of sclerosis and broke this up into *number of segmentally sclerosed glomeruli* and *number of globally sclerosed glomeruli*.

Prompt creation requires creation of a minimal text string that describes the task and includes the report information. Where applicable, related entity types (often those that appear next to each other) are grouped for simultaneous examination in a prompt question. For example, *number of glomeruli*, *number of segmentally sclerosed glomeruli*[2], *number of globally sclerosed glomeruli*, and *presence of abnormal glomeruli* are all related to glomeruli and so can be combined into the same prompt question. These steps culminated in a set of initial annotation rules and guidelines, which were reviewed and iteratively refined in consultation with the

[2]A brief aside on renal biology: the glomerulus is a tuft of capillaries that the blood passes through as part of the renal filtration process. The glomerulus is made up of segments and shaped like an orange - partial or segmental sclerosis refers to cases where segments of the glomerulus have sclerosed, i.e., hardened.

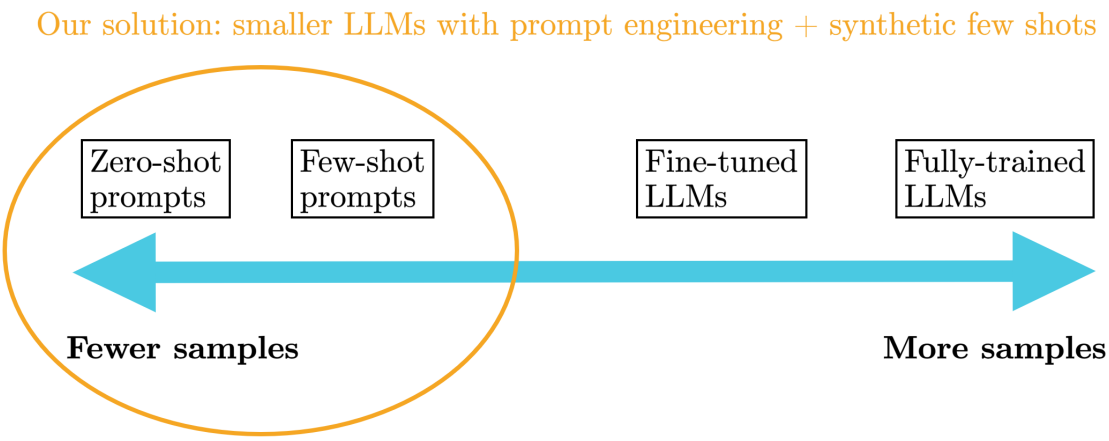



Figure 6: Where our solution fits into the LLM-sample spectrum.

clinician.

Building on questions asked by LJ Miranda [34] while he was developing an NER pipeline for Tagalog [35], this iterative process was guided by four central questions: (1) Are these entities relevant to the biology being studied?, (2) Do entities share characteristics with other entities (e.g. contain similar terms/biology, appear in similar region of report)?, (3) Is it beneficial for the model to learn particular edge cases and can this be encoded as additional prompt information?, (4) Are any guidelines vague or confusing - should we remove or update them?

### 3.4 LLM Modelling and Evaluation

The choice of models prioritised efficient use of resources, balancing computational limitations with the need for robust performance. Although this report is centred on histopathology reports of renal biopsies, our long-term aim is to generalise to more reports in more complex areas of biology and that describe other types of data modalities (discussed in Section **??**). From the challenges of entity variability and relations alongside the aims of a generalisable pipeline, we decided to take a more flexible model class and frame the task as question answering (QA) rather than NER. Given that we have a limited amount of data, we chose to move to modern iterations of autoregressive models (e.g. BERT, GPT, etc.) which can perform well with minimal finetuning or in-context learning.

Given our privacy and infrastructure constraints, we explored smaller generative LLMs on the order of 100M-5B parameters. Our compute is not ideal for supervised fine-tuning though it may be possible - we return to this idea briefly in the discussion. Instead we focus on LLMs which allow input prompts. Our choice of LLM models relative to others is shown in Figure 6. All models were run locally in keeping with our privacy constraints - we have options for two local inferencing servers `Ollama` and `llama-cpp-python`. We use five instruction-tuned models (organised as "(name) (number of parameters) (precision/quantisation)"): *Qwen-2.5 1.5B FP16*, *Gemma 2 2B FP16*, *Llama 3.2 3B Q8*, *Phi-3.5 Mini 3.8B Q4*, *Phi-3.5 Mini 3.8B Q8*. These models were chosen based on if they could run on a 16GB RAM laptop within a reasonable amount of time with priority given to models with less quantisation.

We describe our QA task formally. Let $\mathcal{D} = \{(r_1, a_1), ..., (r_n, a_n)\}$ be a dataset of $n$ pathology reports $r_i$ and their corresponding annotations $a_i$. Each annotation $a_i$ is a set of entity-

| Model Name | # Param. | Quant. | Size | Context Window |
|---|---|---|---|---|
| **Qwen-2.5** | 1.5B | FP16 | 3.1GB | 32K |
| **Gemma 2** | 2B | FP16 | 5.2GB | 8K |
| **Llama 3.2** | 3B | Q8 | 3.4GB | 8K |
| **Phi-3.5 Mini** | 3.8B | Q4 | 2.4GB | 128K |
| **Phi-3.5 Mini** | 3.8B | Q8 | 4.1GB | 128K |

Table 2: Comparison of model specifications including parameter count, quantisation level, model size, and context window length (in tokens).

value pairs $a_i = \{(e_1, v_1), ..., (e_m, v_m)\}$ where $e_j \in \mathcal{E}$ represents an entity from our predefined schema and $v_j \in \mathcal{V}$ is its corresponding value. For each report $r_i$, the task is to learn a function (in this case, a prompt) $f : \mathcal{R} \rightarrow \mathcal{A}$ that maps from the space of reports $\mathcal{R}$ to the space of annotations $\mathcal{A}$, such that: $f(r_i) = \hat{a}_i \approx a_i$ where $\hat{a}_i$ is the predicted annotation.

We evaluate this task under four conditions:

- Zero-shot without guidelines: $f_0(r)$
- Two-shot without guidelines: $f_2(r, \{(r_{e1}^{syn}, a_{e1}^{syn}), (r_{e2}^{syn}, a_{e2}^{syn})\})$
- Zero-shot with guidelines: $f_0^g(r, g)$
- Two-shot with guidelines: $f_2^g(r, \{(r_{e1}^{syn}, a_{e1}^{syn}), (r_{e2}^{syn}, a_{e2}^{syn})\}, g)$

where $g$ represents the annotation guidelines, and $(r_{e1}^{syn}, a_{e1}^{syn}), (r_{e2}^{syn}, a_{e2}^{syn})$ are synthetic exemplar report-annotation pairs. The quality of prediction is measured by accuracy: Accuracy $= \frac{1}{n}\sum_{i=1}^{n} \mathbf{1}[\hat{a}_i = a_i]$, where $\mathbf{1}$ is the indicator function that returns 1 if the predicted annotation matches the ground truth, and 0 otherwise.

The prompt guidelines are refined through repeated iteration of the model and viewing errors. To allow for easy evaluation with our human-annotated reports, the input prompt included a dictionary template of entity codes to specify how the output should look. The output is a string containing a JSON dictionary - we extract the dictionary from the string (important as smaller models occasionally add unnecessary text outside the dictionary itself) and parse it as a JSON. All keys and values in this JSON are strings and so we evaluate them with string-based methods.

For the entity types described in the previous section, we use exact string matching to compare the first four types, and a smaller 1B parameter LLaMA 3.2 model under the LLM-as-a-judge approach to compare whether the *string-complex* entities match. When using this LLM-as-a-Judge (LAAJ) method, we keep it simple by asking true/false questions. We include some simple preprocessing (e.g. turn all None/null/NaN values to 0, making all entities lowercase, defaulting to False in clear mismatch cases) before the LAAJ check to ensure the LAAJ has fewer edge cases. Regardless of choice of inferencing server (we have options for `ollama` and `llama-cpp-python`), the temperature is always zero and the maximum number of tokens is two such that it can output "True" or "False" and no more. All entities are weighted equally for simplicity and so a schema of four entities will have a quarter (1/4) point per entity per report. In practical settings, varied entity weighting might be preferred, but this is sufficient for our proof-of-concept.

As part of reading through the renal biopsy reports, we hand-crafted a glossary of key terms. We converted this to a Python dictionary and built pairs of entities that are "exact" matches

(i.e. the same entity), "synonyms", "similar"-enough, and "different". These are used as test cases to verify our final chosen LLM-as-a-Judge model.

### 3.5 Alternative Models

We experimented with spaCy, BERT models finetuned on the SQuAD dataset [36], NuExtract [16], GLiNER [12] models, with their outputs transformed to the same format as our QA outputs so they could be compared with our LAAJ format for standardisation. All of these models were chosen as they required no training/fine-tuning.

For spaCy, our entity guidelines were transformed into regex and if-else rules to use with spaCy's `custom_entity_ruler` functionality. Default values were set to majority values for entities, e.g., *n_segmental* is almost always 0 in our 100 samples, and so the default value here was 0. Similar situations with *cortex_present* as True, *medulla_present* as False, each of the numerical values as 0 (including *chronic_change*), and *transplant* as False. To deal with the *chronic_change* entity that could be a string or number, we created two entities for the numerical percentage and severity adjective respectively, and set the final *chronic_change* entity to the percentage if present, else adjective.

Two BERT models were chosen as the backbones - a general RoBERTa QA model `deepset/roberta-base-squad2` [37] and one trained initially on biomedical data `dmis-lab/biobert-large-cased-v1.1-squad` [10]. GLiNER was used with a biomedical BERT backbone where we picked the most relevant entity label (as sometimes there were more than one) associated with an entity class. NuExtract as used out-the-box. All three of these model classes (BERT-QA (zero-shot QA), GLiNER (zero-shot NER), NuExtract (zero-shot structured information extraction)) used a slightly modified entity schema where the entities were phrased in an appropriate way for the model, e.g. *n_total* became *How many total glomeruli are there?* in BERT-QA and *Number of total glomeruli* in GLiNER and NuExtract. All three model classes included post-processing steps based on the spaCy rules above to transform our outputs to the required style, and, importantly, set appropriate default values.

### 3.6 Annotation Processes

**Human annotation via Streamlit annotation app.** We have built a Streamlit web app for QA annotation that can be customised to display relevant sections of biomedical reports. The implementation is discussed in Section 3.7 and part of the app is shown in Figure 7. Here, we note down tips that make the annotation process easier. When doing your annotations, it is tempting to start using abbreviations for recurrent phrases (e.g. "chronic allograft nephropathy" as "CAN"). It is better to avoid this as it makes the final comparison between your annotations and the LLM smoother. Once you have 10 reports, we recommend running your LLM model and seeing which entities it does and does not pick up. From this, you can refine the entity schema and prompts, alongside learning the best way to phrase annotations so they align with the LLM outputs (without ruining semantics). A similar app, previously developed by NHSE Data Science, was used for NER annotation.

**Automated annotation via disagreement modelling.** In order to complete the automated annotation, we run two separate LLM models to get two annotated JSON files. We compare these files for matches and mismatches - any reports with mismatches above a certain threshold are flagged for clinician review via an additional entity in the JSON called *clinician_check*. The clinician can then check a box at the top of the Streamlit annotation app to view all these reports or view predictions from both models using the Streamlit comparison app described below. The disagreement threshold should be based on number of entities - if you have 2

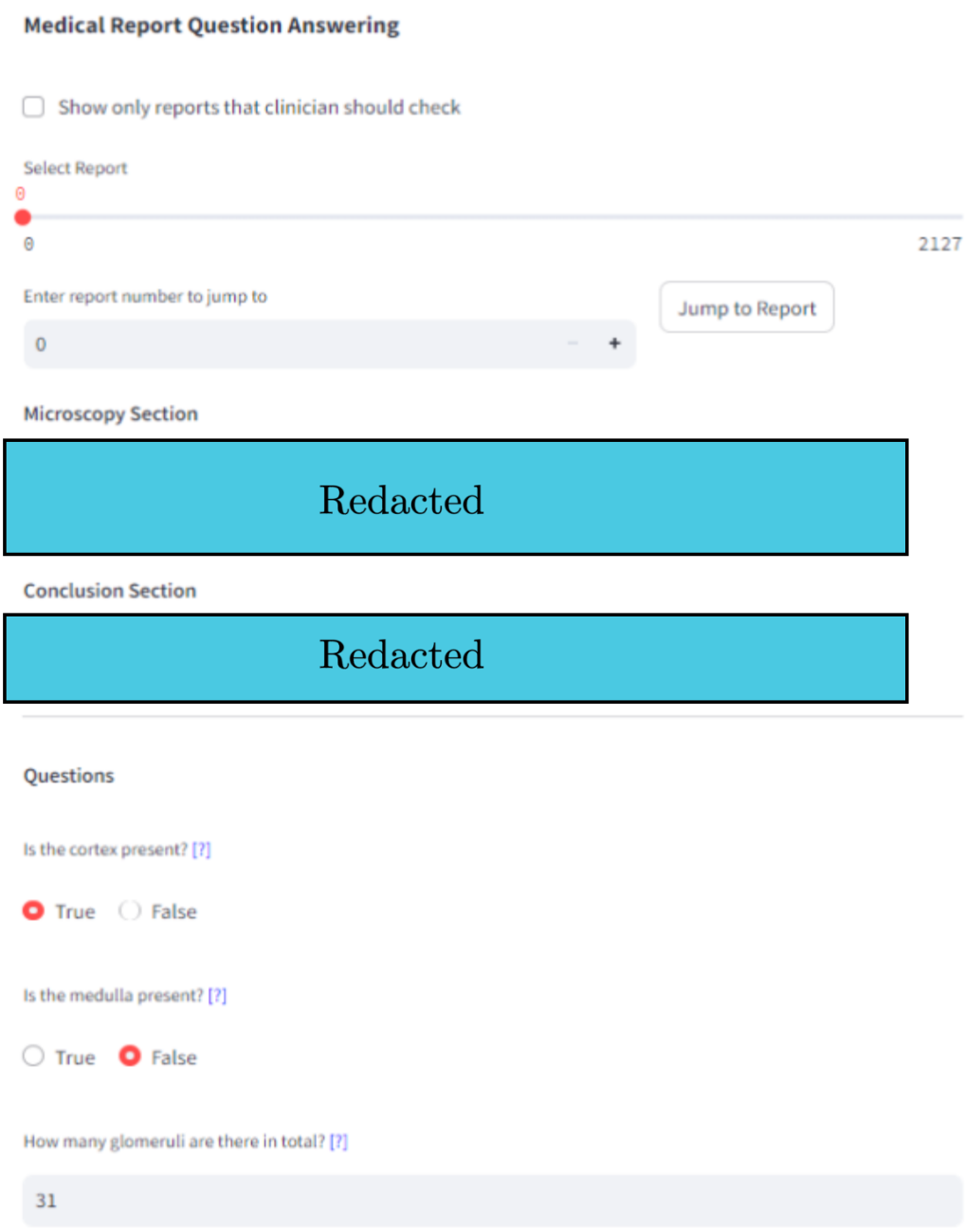



Figure 7: Streamlit app used for QA annotation of reports, displaying three entities: *cortex presence*, *medulla presence*, and *number of total glomeruli*.

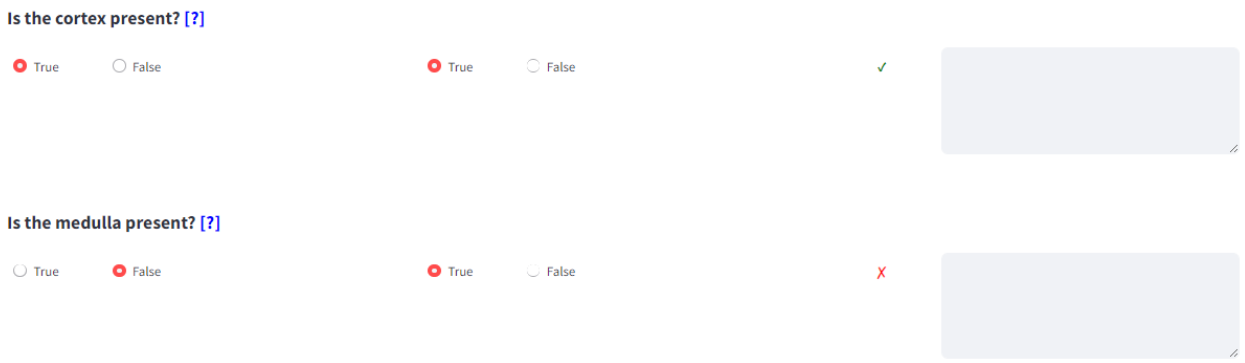



Figure 8: Example report in Streamlit comparison app showing the predictions of two models for *cortex presence* and *medulla presence* entities along with the comment box for qualitative error analysis.

entities, a threshold of 0.5 (1 entity wrong) is probably reasonable, whereas a threshold of 0.3 (3 wrong) would be suitable for a 10-entity schema. For our schema, we notice a 3 entities out of 10 are responsible for the majority of errors and so pick a threshold of 0.3.

**Automated annotation comparison via Streamlit comparison app.** We have designed another Streamlit app to display the predicted results from the automated annotation and add comments on mismatched entities, as shown in Figure 8. The app shows the predictions of both models side-by-side along with scores corresponding to their matches (note: these matches are based on our evaluation methods described in Section 3.4 and are fixed during this process), along with a box to add comments per entity per report, with the option to save these comments for further analysis. Comments should include what the correct answer was and which model predicted wrong - doing this over multiple reports can help build general reasons for errors that can feed back into guideline and prompt creation.

## 3.7 Code Design and Implementation

Our software maps the annotation workflow into three modular code components, centered around a project-specific guidelines spreadsheet, called `guidelines.xlsx`, shown in Table 4. This file defines medical entities/phrases to be found in the reports, their codes, their annotation guidelines, and resulting prompts in a format accessible to non-computational researchers. The three distinct component types balance standardisation with flexibility. Utility components handle common tasks such as processing the `guidelines.xlsx` file and evaluation. Shared components, implemented as base classes, provide the fundamental architecture for question-answering and annotation workflows. Project-specific components, including preprocessing routines, few-shot examples, and text processing logic.

The shared components include: (1) a `qa.py` file which contains the general system message and can be instantiated to implement project-specific question-answering logic with `Ollama` models, and, (2) a Streamlit-based application `qa_app.py` which contains the general app logic for report annotation and can be instantiated to display project-specific reports appropriately. The `guidelines.xlsx` interfaces with these to produce appropriate prompts for your model and display the report text with space for entity annotations in the Streamlit app. Full steps to apply this annotation workflow to your dataset (along with necessary environment package downloads) can be found in the project repository's `README.md`.

| Question To Ask | Entity Type | Entity Guidelines | Entity Code | Combined Prompt Question | Combined Guidelines |
|---|---|---|---|---|---|
| *Initial questions you get from clinicians and refine from reading reports. This is displayed in Streamlit app.* | *Types in order of complexity: boolean, categorical, numerical, string-simple, string-complex. Can also be combinations e.g. numerical/string-simple.* | *Initial guidelines per entity which you get from reading the reports. Note: use " not ' to avoid Streamlit markdown issues.* | *How entity shown in JSON* | *Certain entities might make sense to group together in final prompt. Add combined questions in here and merge cells.* | *Shortened guidelines that link individual entity guidelines together for the combined prompt questions.* |
| Is the cortex present? | boolean | Cortex is the inside of the kidney. If neither medulla or cortex is mentioned, assume the cortex is there and answer True. | cortex_present | Are the medulla and cortex present? | If they are mentioned include them unless explicitly says they're missing. Lack of mention of either implies absence but lack of mention of both implies just cortex. |
| Is the medulla present? | boolean | Medulla is outside of kidney. (The medulla is covered by the capsule.) If it is not mentioned, assume it is not there and answer False. | medulla_present | | |

Table 3: Example of first few rows of `guidelines.xlsx` file showing the questions that practitioners fill out. Note: "boolean" and "Binary" are interchangeable.

# 4 Results

## 4.1 Exploratory Data Analysis

The initial dataset preprocessing yielded 2128 reports for analysis, of which 2111 contained both "MICROSCOPY" and "CONCLUSION" sections as input to our LLM models. Figure 10 illustrates the frequency distribution of terms (including orthographic variations identified through Levenshtein distance calculations) across all reports after removing common stop words. Orange highlights are shown for words associated with our entities. Analysis of report sections revealed that the "MICROSCOPY" section was consistently the longest (Figure 9). Word and character count distributions for the key sections "MICROSCOPY" and "CONCLUSION" are presented in Figure 11. The reports exhibited a mean length of 854 $\pm$ 358 words, ranging from 50 to 3493 words. This maximum length corresponds to approximately 4656 tokens (calculated using the standard 1.33 words-to-token ratio), establishing the minimum required context window for LLM inference.

To assess validity of the clinical renal biopsy pathway outlined in Figure 2, a qualitative examination of diagnoses was conducted on the 100-report subset. Triggers for the biopsies are not explicitly included in these reports so a specific estimate is not possible. Diagnoses include both rejection severity (if transplant biopsy) and final kidney condition. Rejections range from none to grade 0 (borderline, normal), grade 1 (mild, moderate), grade 2 (chronic, significant), and grade 3. There are multiple different conditions (including their synonyms) mentioned in more than one patient: chronic vasculopathy, BK virus, chronic allograft nephropathy (mentioned much more than other conditions), tubular necrosis, acute pyelonephritis, urinary tract infection, cortical necrosis, lupus nephritis, and varying forms of glomerulonephritis (e.g. focal segmental, membranous). There are also several mentioned once: cystic hamartoma of renal pelvi, tubulointerstitial nephritis, parenchymal infarction, and renal artery thrombosis. There are 13 unique conditions with all grades of rejection reported.

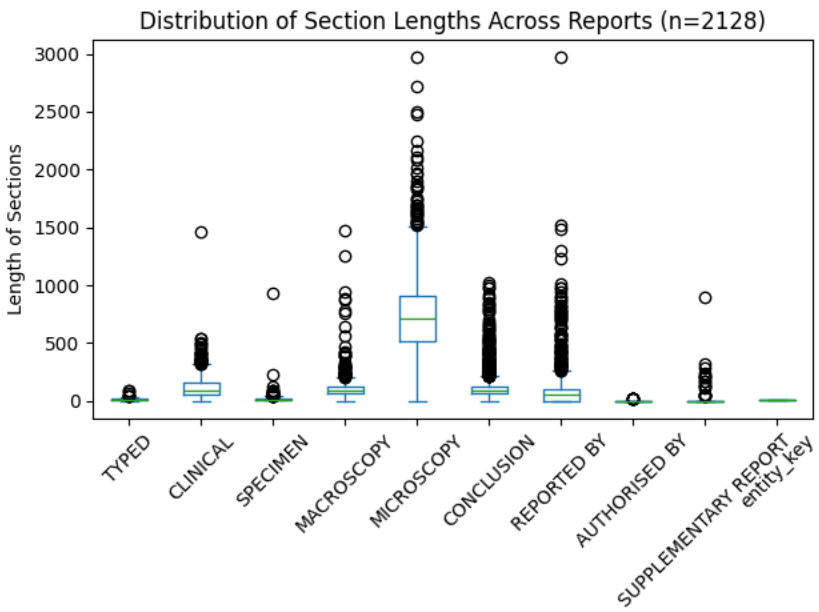



Figure 9: Boxplot showing lengths of each section of the report.

## 4.2 Final Entity Guidelines and Prompt Creation

We read over a sample of 100 reports, producing a set of entity guidelines (shown in Table 4 with colours) for use in our input prompts to constrain the model better. We devised a set of general rules for modifying entities and creating their guidelines: (1), data types can change (e.g. from Binary to Categorical), (2), guidelines should include how to deal with a case where the entity (or set of entities) is not mentioned, (3), guidelines should include the scope of entities with respect to broader biology, (4), related to point (3), guidelines should include phrases that definitely do or do not describe an entity, and (5), related to (1) on data type changes, if categorisation is present or needed, should the guidelines include a mapping from categories to severity scores (i.e. are the variables ordinal)? We describe our changes from our initial entities in Table 1 with these guidelines.

For (1), *cortex presence* and *medulla presence* have been combined into one entity (and subsequent question) since they almost always appear in the same sentence. We have split *number of sclerosed glomeruli* into *number of segmentally sclerosed glomeruli* and *number of globally sclerosed glomeruli* to deal with added granularity in the reports that is relevant for final diagnosis as some conditions solely deal with global sclerosis. The data type for *percentage of chronic change/fibrosis* has been changed to include a string given that adjectives are used to describe the condition more often than numbers. We do not perform any entity linking to codes and so *final diagnosis* has been changed to complex string. For (5), our guidelines include an adjective-to-number schema for *percentage of chronic change/fibrosis* and inclusion of rejection severity for the *final diagnosis*.

For (2), we add guidelines for the appropriate annotation when entities are not mentioned, including edge cases for grouped entities. For (3), we describe broad biological scope: we are focussed on simpler generalisable biology and so edge cases like partially-observed glomeruli and rarer sclerotic lesions are ignored. We also make sure that *abnormal glomeruli present* includes guidelines that do not overlap with sclerotic abnormalities to keep the schema entities clearer. For (4), we describe the definition and specific phrases entailing an entity's presence or lack of presence.

An example one-shot prompt for two grouped entities - *cortex presence* and *medulla presence* -

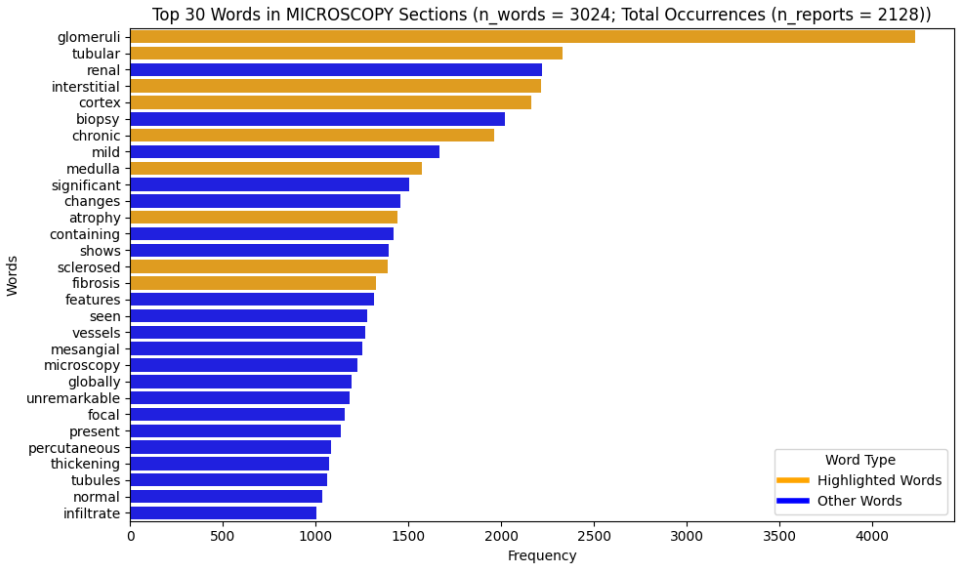



Figure 10: Top 30 words in "MICROSCOPY" section across reports.

is shown in Figure 12. The template starts with a system message and task description as per other common prompt-based NER methods like GPT-NER [13], displays the grouped entity questions and grouped entity guidelines from our `guidelines.xlsx` file, and then shows an example report with its answer followed by the real report. The real prompt includes all entities and both the "MICROSCOPY" and "CONCLUSION" sections.

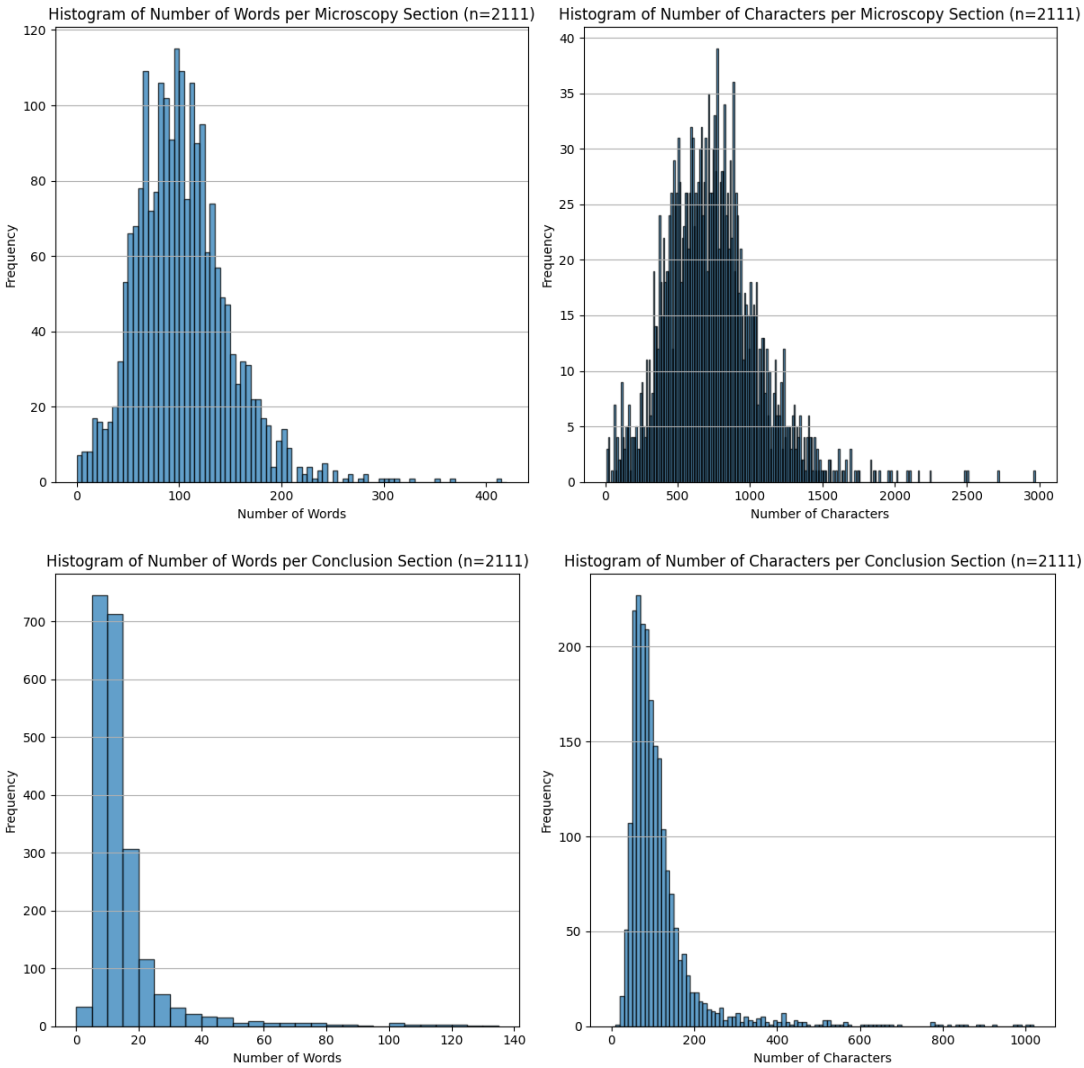



Figure 11: Histograms of number of words (left) and characters (right) for the microscopy (top) and conclusion (bottom) sections. There are fewer reports here than in Figures 9 and 10 since not all reports with "MICROSCOPY" sections had "CONCLUSION" sections.

| Entity type | Data type | Guidelines |
|---|---|---|
| cortex / medulla presence | Tuple(Binary, Binary) | - Lack of mention of either implies absence. But if neither mentioned, assume cortex. |
| number of glomeruli | Numerical | - All we care about is the total number of complete glomeruli mentioned. If no reference is made to "complete glomeruli," assume they are complete.<br>- Cores and sections are not relevant to calculations of glomeruli. |
| number of globally and locally sclerosed glomeruli | Tuple(Numerical, Numerical) | - Focusing on standard sclerotic lesions for now. (There are rarer ones like necrotising lesions.) |
| abnormal glomeruli present | Binary | - Only consider non-sclerotic abnormalities.<br>- There are phrases for normal: normal, no abnormalities, unremarkable.<br>- There are phrases for abnormalities: changes in size and shape, non-sclerotic glomeruli changes. |
| % chronic change/fibrosis | Numerical or String-Simple | - What is chronic change? Chronic change = tubular atrophy + interstitial fibrosis. Can write as 3-tuple: atrophy (y/n), fibrosis (y/n), % chronic change.<br>- What is chronic change not? Chronic changes in infiltrate are related to transplant biopsies and not relevant here.<br>- Have schema linking adjectives to %, e.g. 5-10% = "minimal"/"small", 75-100% = "florid"/"complete" |
| final diagnosis | String-Complex | - Identify if transplant or native biopsy. (Transplants mention things related to white blood cells because of immune response after rejection. Native biopsies talk about abnormal glomeruli if something bad.)<br>- If transplant, report the grade/quantifier word associated with severity of rejection.<br>- Report the final diagnosis of the condition. If no other diagnosis is given, report the rejection diagnosis, and if that isn't there, say [missing]. |

Table 4: Standard rules/principles to think about when creating entity guidelines: data types can change, lack of mention, scope of entity, phrases for/against, severity scores. Full version of guidelines available in the `guidelines.xlsx` file.

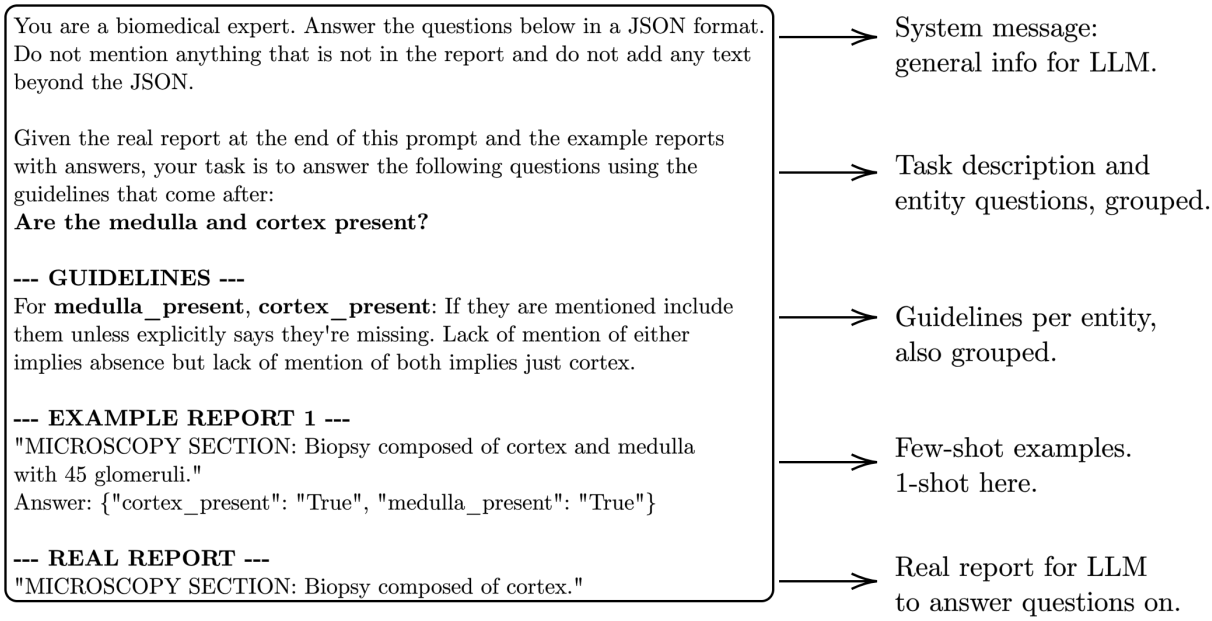



Figure 12: Example prompt for two entities.

## 4.3 Refining the LLM-as-a-Judge Function

We create multiple pairs of output renal biopsy phrases for the following categories: "exact", "same concept", "similar enough", and "different" categories. "Exact" pairs are the exactly same phrase (this is a sanity check for the model), "same concept" have the exact same meaning but are phrased differently (including abbreviations), "similar enough" are phrases that are similar enough to be considered the same for our purposes (requiring some inference from the model), and "different" are phrases which are not the same. We include edge cases from running our model over *string* entities from our guidelines. We test the ability for booleans to be semantically exact but syntactically different, e.g., "True" vs "Present" for a entity's presence, and do not allow any margin of error for numbers e.g. "14" cannot be accepted for "15".

We test our LAAJ function (the model used and prompt) via scores for "symmetry", "expert agreement", and "consistency". For "symmetry", we run our entity pair as input to our prompt with inputs as $(entity_1, entity_2)$ (forward pass) and then $(entity_2, entity_1)$ (backward pass) to confirm the outputs are exactly the same. For "expert agreement", we check that our forward pass outputs the same boolean for similarity as the human ("exact", "synonyms", and "similar" are *True*, "different" is *False*), and, for "consistency", we check both the forward and backward pass output the same boolean for similarity as the human. LLM temperature is always set to zero (though this does not mean completely deterministic results) and so only one trial run per entity pair and score is performed. The aim here is not to achieve perfect performance but to figure out cases in which our LLM evaluator outputs something different to what we want so these cases can be avoided during annotation or given to a clinician for further checks.

### 4.3.1 Initial Versions

**Note: these were not used for final evaluation.**

A version with HuggingFace sentence transformers and cosine similarity was tried initially

but setting the threshold proved to be too difficult, particularly for very differently-phrased synonyms, similar-looking but contradictory phrases, and negations.

Longer prompts were tried with techniques similar to the input prompt such as few-shots and longer sets of guidelines about different comparisons. However, simplicity seemed to be key to good performance on our examples. We found that a generic example for severity improved performance. We show three slight variations of a simple prompt in Figure 13 and their performance using Gemma2 2B FP16 in Figure 15 and Qwen 2.5 1.5B FP16 in Figure 14. We found that Qwen 2.5 1.5B FP16 with Query v3 gave us the best overall performance and this was used for model evaluation in later sections.

We note that Query v1 for Gemma2 looks good for the first three categories but the poor performance on the "different" examples made it too unreliable. We tried some of the other versions - at most, we found a decrease in performance of 3% performance, and so around 30/900 outputs changing, or more accurately, 30/200 given that there were 200 string entities that require LAAJ evaluation. These were mostly nested phrases for the *final diagnosis* entity that were poorly predicted by one prompt. The variation in symmetry is surprising and evidence that this workflow is not yet suitable for deployment workflows. Worse still, these prompts seemed to give far too many false positives when evaluating our LLM predictions in Section 4.5. We tried a different set of prompts shown in the following section and found more accurate downstream performance.

Do the phrases "{phrase 1}" and "{phrase 2}" describe similar concepts? Only answer True or False. Allow some deviation in phrasing.

Query v1: simple question

Do the phrases "{phrase 1}" and "{phrase 2}" describe similar concepts? Only answer True or False.

Allow some deviation in phrasing but they are NOT similar if their severity is different e.g. "severe x" vs "mild x" should output False.

Query v2: simple question + severity example

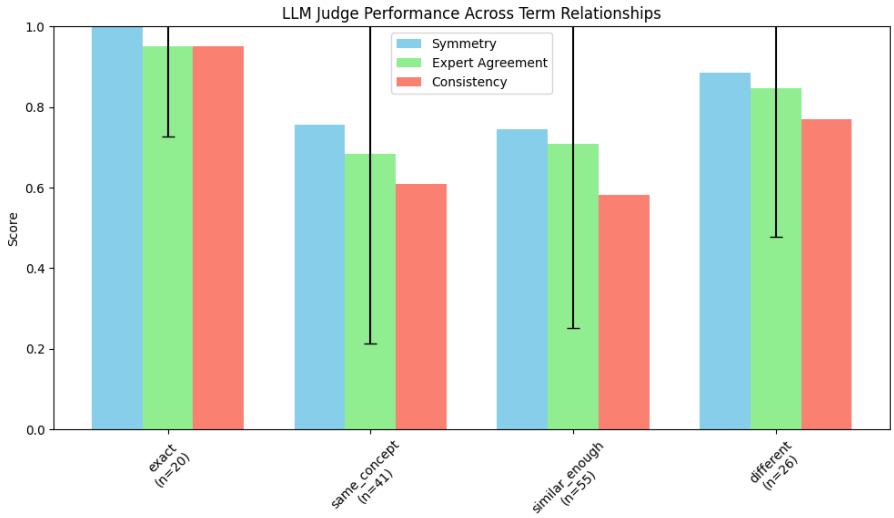
Do the phrases "{phrase 1}" and "{phrase 2}" describe similar concepts? Only answer True or False.

Allow some deviation in phrasing but they are NOT similar if their severity is different e.g. "severe x" vs "mild x" should output False.

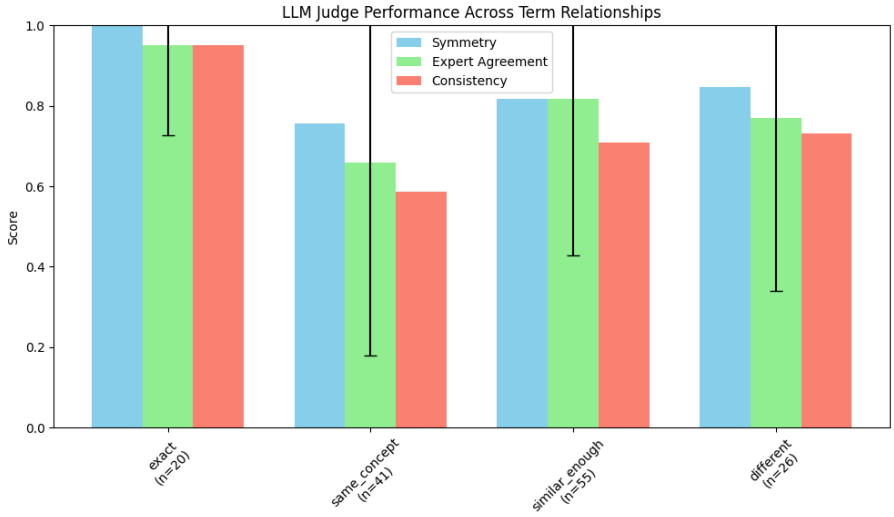
If "None", "NaN", or "null" is mentioned, output False.

Query v3: simple question + severity example + null check

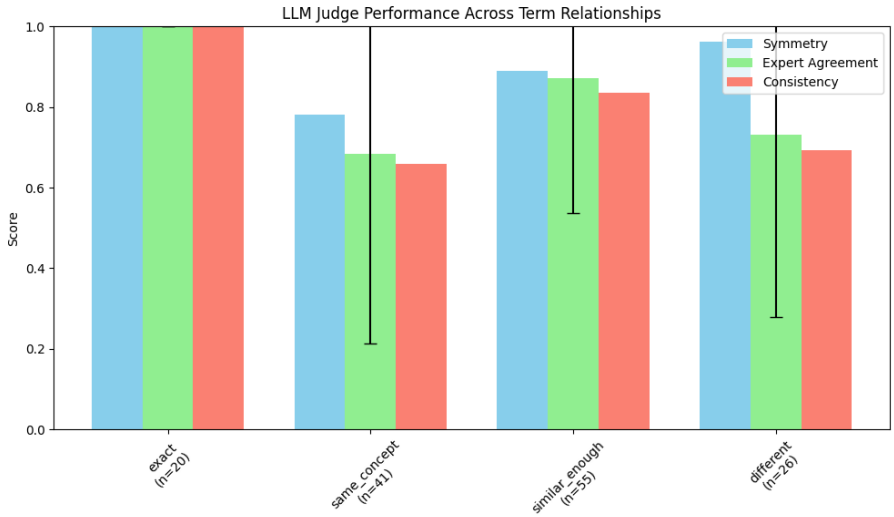
Figure 13: Development of final LLM-as-a-Judge (LAAJ) query.




(a) Query v1: simple question

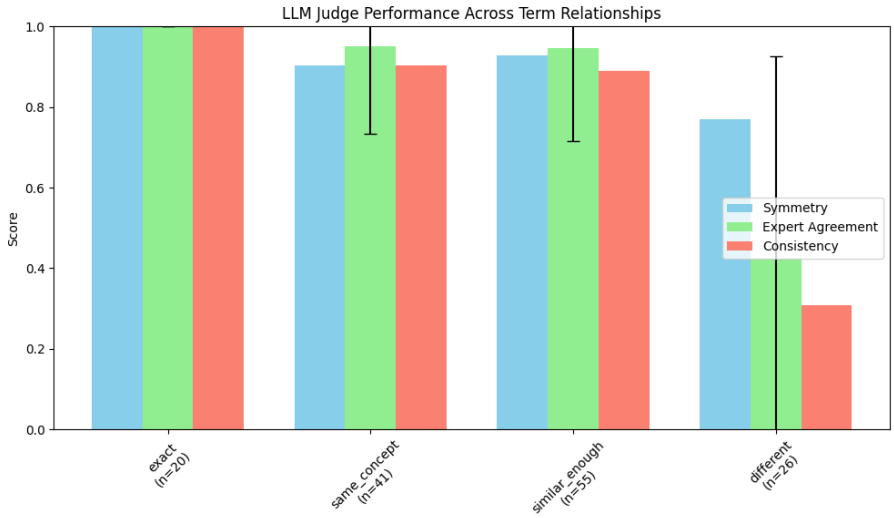



(b) Query v2: simple question + severity example

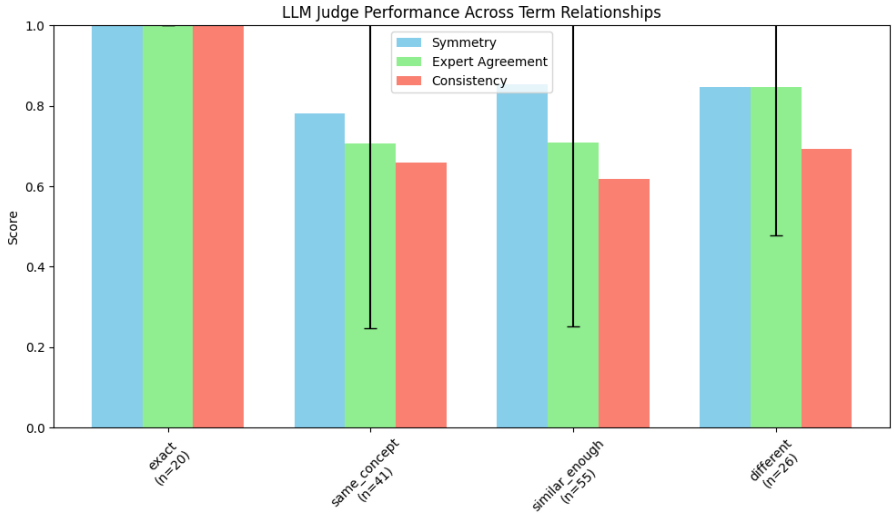



(c) Query v3: simple question + severity example + null check

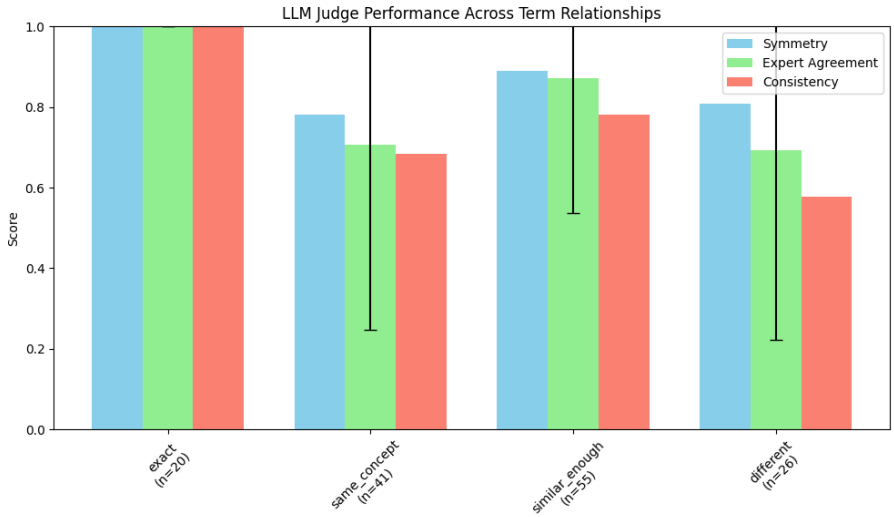
Figure 14: Results from Qwen FP16 model across different LAAJ queries. For each plot, from left to right: exact, same_concept, similar_enough, different.




(a) Query v1: simple question




(b) Query v2: simple question + severity example




(c) Query v3: simple question + severity example + null check

Figure 15: Results from Gemma2 2B FP16 model across different LAAJ queries. For each plot, from left to right: exact, same_concept, similar_enough, different.

### 4.3.2 Final Version

| | |
|---|---|
| Are the phrases "{entity1}" and "{entity2}" describing equivalent or similar concepts? Only answer True or False. | Query v1: simple question |
| You are a renal biopsy expert.<br><br>Are the phrases "{entity1}" and "{entity2}" describing equivalent or similar concepts? Only answer True or False. | Query v2: simple question + expert affirmation / bio constraint |
| You are a renal biopsy expert.<br><br>Are the phrases "{entity1}" and "{entity2}" describing equivalent or similar concepts? Only answer True or False.<br><br>Answer based on the nouns, adjectives, or numbers. | Query v3: simple question + expert affirmation / bio constraint + answer constraint |
| [null checks, force lowercase, exact string matching]<br><br>You are a renal biopsy expert.<br><br>Are the phrases "{entity1}" and "{entity2}" describing equivalent or similar concepts? Only answer True or False.<br><br>Answer based on the nouns, adjectives, or numbers. | Query v4: simple question + expert affirmation / bio constraint + answer constraint + post-annotation checks and formatting |

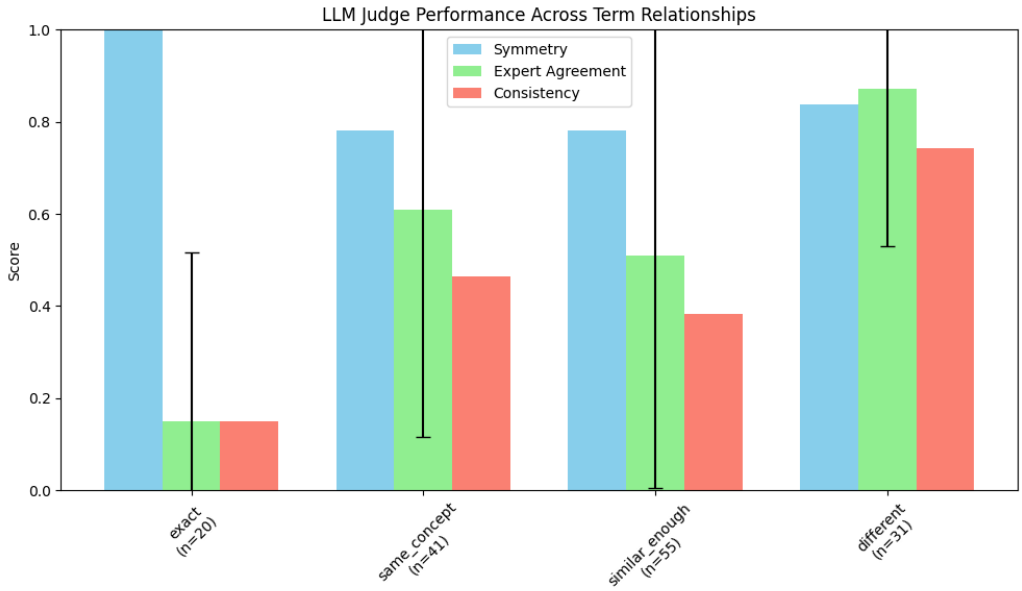
Figure 16: Development of final LLM-as-a-Judge (LAAJ) query.

One issue with the previous set of prompts is it would either perform well on the *chronic_change* or *diagnosis* entity but not both. We were able to create our final prompt in Figure 16 that balanced the two with reference to "equivalent or similar concepts", appropriate use of expert affirmations, answer constraints, and if-else statements for edge cases. **We note that this prompt is not perfect - it does not always evaluate as we would like it to (gets it wrong roughly 10% of the time on the *string* entities) - and so should be used with care.**

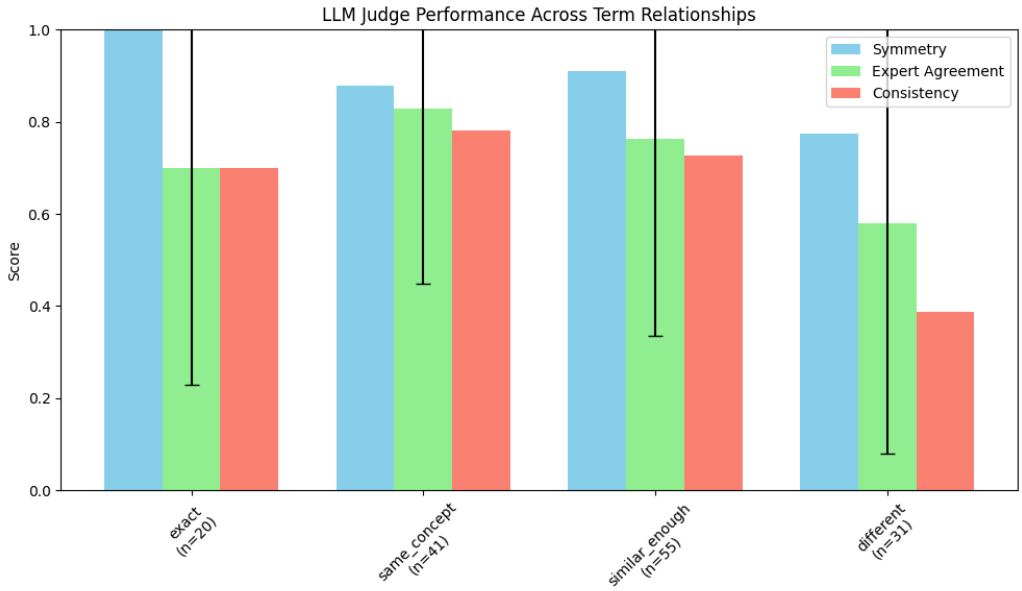
We show results for this prompt on our test cases with Llama 3.2 3B FP16 in Figure 17. The initial simple query was good at spotting differences but poor at all synonyms and extremely poor at exact matches - the symmetry was still high so it appears as though it was defaulting to False for many of these. Inclusion of the expert affirmation made performance much more consistent across the four categories. Inclusion of the answer constraints improved the same_concept and similar_enough categories but decreased the performance on differences, implying it was defaulting to True. A final set of if-else checks for edge cases was able to bring performance up for the exact matches. The remaining issues for difference include (1) proper handling of "null/None/NaN" values, (2) subtle rejection differences (Rejection Grade 1A vs 1B, T-cell mediated rejection vs antibody-mediated rejection), (3) the inability to separate different types of sclerosis (global vs segmental, diffuse vs focal), and (4) the occasional confusion of opposite/orthogonal terms (cortex vs medulla, interstitial vs scattered, acute rejection vs chronic rejection).

The first three issues are understandable, the last indicates much more work is needed for a generalisable LAAJ prompt but fortunately, these are not issues that ever happen in actual annotation. In fact, we found that this prompt gave more accurate evaluation than our previous set of queries even though it was slightly worse in overall performance. We assume this is because our set of test pairs cover much broader ground than the actual LLM vs human

pairs for our two string entities *chronic_change* and *diagnosis*; in other words, the previous prompt might be better for general renal biopsy schemas but this one is better for our entity schema. Future work could look to build test sets that are more targeted to a specific entity schema.

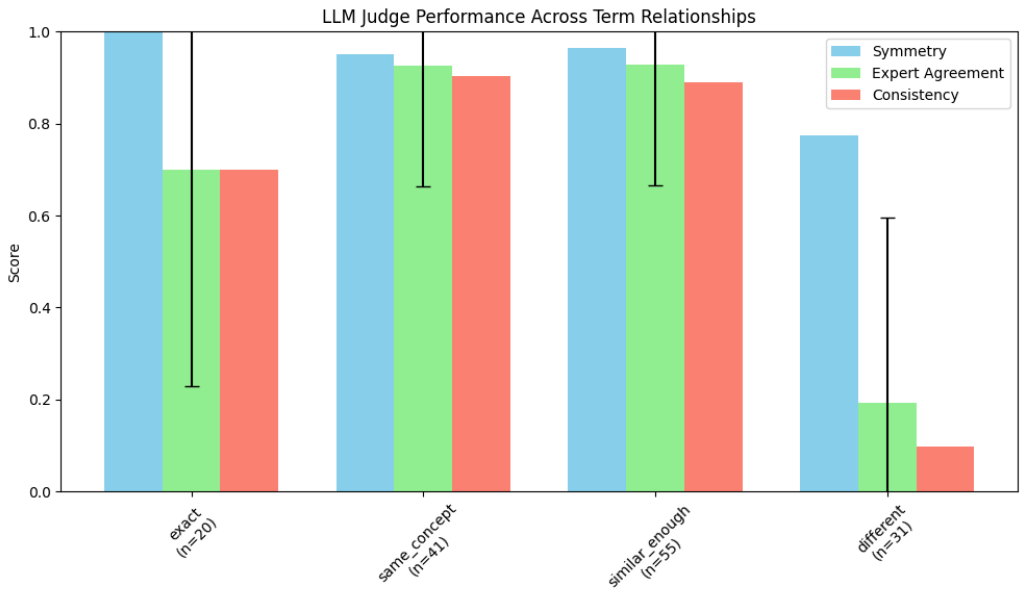



(a) Query v1: simple question

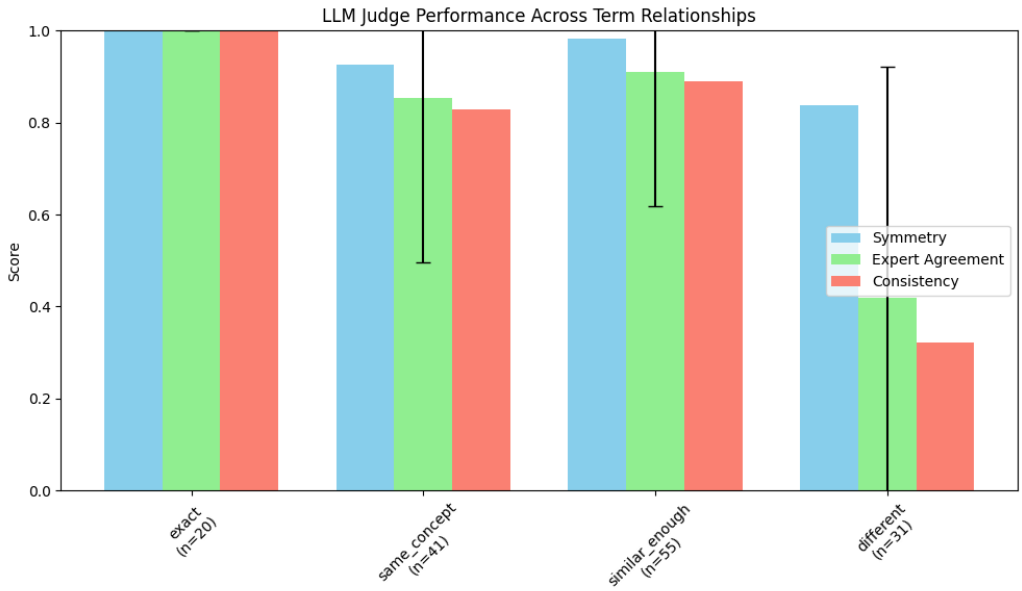



(b) Query v2: simple question + expert affirmation

Figure 17: Results from Llama 3.2 3B FP16 across different LAAJ queries. For each plot, from left to right: exact, same_concept, similar_enough, different. [Continued below...]




(a) Query v3: simple question + expert affirmation + answer constraints




(b) Query v4: simple question + expert affirmation + answer constraints + post-annotation checks

Figure 17: Continued from above.

## 4.4 Prototyping Alternative Models

We annotated 100 reports using our Streamlit app. The resulting human-annotated JSON was compared to annotations from spaCy, BERT models fine-tuned on the SQuAD dataset, GLiNER, and NuExtract in Table 5. spaCy showed the best total accuracy of 68.9%. It showed strong performance (>80%) for the three simpler boolean entities: *cortex present*, *medulla present*, and *transplant*, and also achieved 95% accuracy on the *number of segmentally sclerosed glomeruli* entity, the highest of any models including the LLMs in the next section. The BioBERT-SQuAD and RoBERTa-SQuAD models showed total performance of 62.6% and 59.0% respectively, with RoBERTa stronger on average across the numerical entities and BioBERT stronger on all entities reliant on finding biomedical text. The zero-shot NER model, GLiNER (with biomedical BERT backbone), was used with a threshold of 0.3, and achieved total performance of 62.0%. All models bar BioBERT performed poorly on the *diagnosis* entity with spaCy unable to annotate any. spaCy was the quickest model, annotating our entire 100 report sample in just two seconds. GLiNER, RoBERTa-SQuAD, and BioBERT-SQuAD were all able to annotate fairly quickly too, each taking between 2-16 mins for complete sample annotation. NuExtract took over 20 minutes per report and we ran out of time before being able to annotate the entire sample, however, initial results on 20 reports were promising and comparable to the best-performing LLMs in the next section.

| **Model →**<br>**Entity ↓** | **spaCy** | **BioBERT-SQuAD** | **RoBERTa-SQuAD** | **GLiNER** | **NuExtract** |
|---|---|---|---|---|---|
| **cortex_present** | 95/100 | 96/100 | 94/100 | 52/100 | 17/20 |
| **medulla_present** | 98/100 | 88/100 | 74/100 | 88/100 | 18/20 |
| **n_total** | 56/100 | 80/100 | 66/100 | 81/100 | 19/20 |
| **n_segmental** | 95/100 | 45/100 | 67/100 | 91/100 | 17/20 |
| **n_global** | 71/100 | 45/100 | 75/100 | 70/100 | 18/20 |
| **abnormal_glomeruli** | 53/100 | 50/100 | 53/100 | 54/100 | 14/20 |
| **chronic_change** | 69/100 | 60/100 | 55/100 | 45/100 | 15/20 |
| **transplant** | 83/100 | 34/100 | 26/100 | 72/100 | 13/20 |
| **diagnosis** | 0/100 | 65/100 | 17/100 | 4/100 | 18/20 |
| **Overall ↓** | | | | | |
| **all entities** | 620/900 | 563/900 | 527/900 | 558/900 | 149 /180 |
| **accuracy** | 68.9% | 62.6% | 58.6% | 62.0% | 82.8% |
| **annotation speed** | 0.02s/report | 9.18s/report | 2.59s/report | 1.78s/report | 1473s/report |

Table 5: Entity Extraction Performance for models that are not generative LLMs. NuExtract results have been highlighted orange since they are incomplete.

## 4.5 Varying Prompts: Guideline Inclusion & Few-Shot Samples

Using the same 100 annotated reports, Table 6 shows accuracies and iteration times per report for our different LLM models. All models with guidelines in the prompt achieved greater than our pre-specified "acceptable" performance threshold of $\geq 70\%$ and are able to run over our complete dataset of 2111 reports within two days, achieving our aim for "reasonable" inference time on GOSH infrastructure. Evaluation times are consistent across all models since LLM-as-a-judge uses the same LLM; this took 3.4s/report and so would take just under two hours for all 2111 reports.

Table 7 shows the results per entity for prompts with two-shot samples and guidelines. All models show ≥80% performance on *cortex present*, *medulla present*, and *number of total glomeruli* entities, and 75-80% performance for the *diagnosis* entity. Qwen 1.5 has the lowest total performance. Both Phi 3.5 models perform >90% on *number of globally sclerosed glomeruli* and *number of segmentally sclerosed glomeruli* entities. On average, *abnormal glomeruli* and *chronic change* are the most challenging entities for these LLMs.

Parsing errors vary in their ease to be automatically corrected. The majority of JSON parsing errors from the LLM's generated string would be a repeated error throughout the reports. For example, the *10 in Phi-3.5 Q8's zero-shot with guidelines cell was due to the LLM adding commented-out explanations for certain "undefined" (where the entity was present value unknown) entities. Similarly, the same model's and Llama 3.2's zero-shot without guidelines cells contain *11 and *43 errors respectively which were all cases where the final curly bracket of the JSON was missing. However, for the Phi 3.5 Q4 model without guidelines, the zero-shot case got stuck on one report for over two hours and so was terminated, and the two-shot case stopped producing JSON answers at the 48th report (the score here reflects this), instead outputting an incoherent story interspersed with Chinese characters. The former two are not ideal but can be corrected fairly easily, while the latter two issues prevent use of that model. Strangely, the Q4 model, after removing parsing errors, outperforms its Q8 sibling and is the top-performing model alongside Gemma 2.

| **Prompt Format →**<br>**Model Name ↓** | **Zero-shot without guidelines** | **Two-shot without guidelines** | **Zero-shot with guidelines** | **Two-shot with guidelines** |
|---|---|---|---|---|
| **Qwen-2.5**<br>1.5B, FP16 | 65.8%<br>19.2s/report | 71.9%<br>18.6s/report | 75.1%<br>20.4s/report | 71.2%[*1]<br>19.9s/report |
| **Gemma 2**<br>2B, FP16 | 72.5%<br>28.2s/report | 81.3%<br>29.4s/report | 80.8%<br>25.8s/report | 82.2%<br>30.0s/report |
| **Llama 3.2**<br>3B, Q8 | 74.8%[*43]<br>19.8s/report | 80.4%[*2]<br>19.2s/report | 79.7%[*20]<br>19.8s/report | 81.7%<br>21.0s/report |
| **Phi-3.5 Mini**<br>3.8B, Q4 | N/A<br>N/A | 36.1%[*35]<br>45.6s/report | 79.0%[*11]<br>33s/report | 82.2%[*4]<br>35.4s/report |
| **Phi-3.5 Mini**<br>3.8B, Q8 | 69.5%[*11]<br>31.8s/report | 78.4%[*4]<br>37.8s/report | 78.7%[*10]<br>40.2s/report | 79.9%[*1]<br>42.1s/report |

Table 6: Table comparing accuracy and report iteration times for different models with zero-shot and two-shot prompts over 100 annotated reports. *x signifies the number of alterations required to allow smooth JSON parsing from the LLM's generated string.

| Model → Entity ↓ | Qwen1.5 | Gemma2 | Llama3.2 | Phi3.5 Q4 | Phi3.5 Q8 |
|---|---|---|---|---|---|
| **cortex_present** | 93/100 | 95/100 | 95/100 | 97/100 | 95/100 |
| **medulla_present** | 85/100 | 80/100 | 86/100 | 85/100 | 85/100 |
| **n_total** | 96/100 | 94/100 | 95/100 | 96/100 | 95/100 |
| **n_segmental** | 69/100 | 83/100 | 69/100 | 91/100 | 90/100 |
| **n_global** | 54/100 | 89/100 | 92/100 | 92/100 | 92/100 |
| **abnormal_glomeruli** | 62/100 | 75/100 | 72/100 | 69/100 | 63/100 |
| **chronic_change** | 52/100 | 72/100 | 77/100 | 51/100 | 47/100 |
| **transplant** | 55/100 | 72/100 | 73/100 | 83/100 | 77/100 |
| **diagnosis** | 75/100 | 80/100 | 76/100 | 76/100 | 75/100 |
| **Overall ↓** | | | | | |
| **all entities** | 641/900 | 749/900 | 735/900 | 740/900 | 719/900 |
| **accuracy** | 71.2% | 82.2% | 81.7% | 82.2% | 79.9% |
| **annotation speed** | 19.9s/report | 30.0s/report | 21.0s/report | 35.4s/report | 42.1s/report |

Table 7: Entity Extraction Performance per LLM model for two-shot prompts with guidelines (same results as final column of Table 4).

## 4.6 Qualitative Evaluation of Disagreement Modelling

**[REDACTED]**

## 4.7 Full Annotation of Corpus

**[REDACTED]**

# 5 Discussion

## 5.1 Overview of Results

### 5.1.1 Alternative Approaches

**NER annotation workflow.** We initially tried NER annotation (via our Streamlit app) which offered some advantages over the QA approach, mainly in its ability to capture precise spans consistently for entities such as numbers and words common to simpler entities like *cortex_presence* and *medulla_presence*. However, the span-based focus presents challenges for some entities, particularly when single spans could correspond to multiple entities. For example, the phrase "there are 10 glomeruli" with no other reference to glomeruli tells you, with some reasoning, all the answers to our glomeruli questions - there are 10 total, 0 globally sclerosed, 0 segmentally sclerosed, and 0 non-sclerotic abnormalities. It is unclear if we should put the same span for each of these entities. We felt the QA schema lent itself more naturally to this type of problem, allowing shorter answers (10, 0, 0, 0) which makes final evaluation possible with simple string matching (since all outputs are strings from the LLM).

After discussion with the clinician on appropriate annotations, we realised even simple entities like binary variables did not always have explicit answers and sometimes required inference from contextual information. Meta-annotations, including qualifiers and negations, posed additional challenges due to their unique phrasing (often varied adjectives which we could not fully cover with, e.g., spaCy regex rules) and the requirement for full span capture. Discontinuous entities like *percentage of chronic change* could be handled through multiple phrases per entity type per report. For example, adjectives can be captured corresponding to different fibrotic symptoms such as "mild" and "moderate" for "mild tubular atrophy and moderate interstitial fibrosis". However, the fundamental question of defining appropriate spans per entity remained hard to formalise into a set of rules. Instead, the QA framework integrates a flexible entity schema which can be changed via natural language, and so is also appropriate for non-data scientist annotators. For these reasons, we believe the QA annotation framework is more suitable than the NER framework.

**Performance.** Baseline approaches, such as spaCy with rules and regex-based extraction, were our initial starting point. A key benefit of spaCy from the annotator's point-of-view is that is forces you to consider default (i.e. majority) values per entity and possible classifications for regex. This works as a great preliminary annotation strategy that feeds nicely into the post-processing of all other models and guideline creation of our main generative LLM workflow. spaCy models fared surprisingly well relative to other models due to their ability to capture all possible "seen" cases via regex, though they struggled with *string*-based entities and co-referencing for *number of globally sclerosed glomeruli*. Often, this is phrased in the report as "There are 10 glomeruli, of which 3 have global sclerosis" - a window around the *total number of glomeruli* entity was required here. Some spaCy results seem surprisingly good but they are often simpler cases where defaulting to one answer is successful. Specifically, *number of segmentally sclerosed glomeruli* (95/100 are zero), *transplant* (74/100 are True), and *chronic_change* (43/100 are 0%). We can use these defaults in post-processing only if the initial models have left the entity unanswered, which spaCy often does, but other models do not.

BioBERT-SQuAD was the best at *diagnosis* by far, much better than RoBERTa-SQuAD, and so likely due to the fact it has been trained on biomedical strings. spaCy returned null for all diagnoses and so scored 0/100. We considered setting a default string value for this entity -

for example, finding the rejection severity by using a window around the word "rejection" - but the phrasing of rejection and diagnosis description varied a lot, even in this constrained renal biopsy setting. GLiNER performs comparably to the BERT models which makes sense given its BERT backbone, though, surprisingly, does not perform well on the *diagnosis* entity. The NuExtract model is a fine-tuned version of Microsoft's Phi-3 model on structured JSON data - it performs comparably to our best LLMs with much less preprocessing (no requirement for guidelines or few-shots), however, its longer inference times prevent us from using it more.

Entities can also vary widely in how they are phrased and may change data type (mentioned as a text string or number), and how long answer spans can be, particularly for our *final diagnosis* entity. Another model we tried but were unable to get good results for was MedCAT. We used the unsupervised version of MedCAT to handle our unlabelled reports and found that the symptoms in our reports were too finegrained to be picked up by the model. MedCAT performed slightly better than spaCy on the *final diagnosis* but it was only able to find 2-3 diseases out of the common 15 for renal biopsies. The *final diagnosis* entity is challenging as it contains both the final diagnosis and rejection status with grade e.g. "Features suggestive of chronic tubulointerstitial nephritis. Evidence of slight rejection (Grade 1a)." If negation is present, it is solely on the rejection part (e.g. "There is no evidence of rejection") but since the model does not include "rejection" as an entity, it does not pick this up. Similarly, nor can it pick up the variations in grade of the rejection.

**Annotation Times and Utility**. spaCy allows you to iterate quickly over the whole dataset and so we would recommend it as an initial starting point for any annotation pipeline, regardless of hardware capability. BERT models are also fairly quick - we would recommend using biomedical models like BioBERT-SQuAD on your string-based entities (the entities that spaCy is worse at) to see how high your overall performance can be with a combination of models. GLiNER takes a similar amount of time as the BERT-QA models but does not produce any large differences in performance. NuExtract models take much longer and are unsuitable for our hardware constraints. Overall, we recommend using spaCy for as many entities as possible, then using a BioBERT-SQuAD model for string-based entities. At this point, if performance is still insufficient, you can move onto the generative LLM models.

### 5.1.2 Annotation Workflow

**Exploratory Data Analysis.** As a sanity check, we see the main words are associated with our highlights. The real number of renal biopsy triggers and diagnoses is slightly more complicated than the general diagram shown in Figure 2 since rejection and diagnoses must be viewed together rather than separately - hence diagnoses are tuples which increases the number of unique values. Based on qualitative analysis of our 100 sample reports, we observed that most of them included (or implicitly mentioned) answers to every entity. We view this as confirmation of appropriate scope for our entity schema. The choice of 100 samples proved to be a reasonable size for qualitative analysis of errors when tweaking prompts and LAAJ functionality, and large enough to view differences between the other models and ours. In hindsight, 50 samples may have been enough and allowed for quicker iteration time.

**QA performance.** The generative LLM approaches performed much better beyond 1B parameters - smaller 0.36B, 0.5B, and 1B parameter models were tried with limited success due to parsing issues, even with inclusion of few-shots and guidelines. However, above 1B, models were able to, sometimes without few-shots or guidelines, perform reasonably well ($> 70\%$) with limited parsing issues. For automated workflows, any number of parsing is-

sues is too many and so FP16 models are preferred. Our top-performing LLM models were able to reach 82.2% accuracy. The use of guidelines helped standardise performance across all models, increasing it by up to 9% from the zero-shot-no-guidelines baseline, and reduce the number of parsing errors. Similarly, inclusion of two in-context examples improved performance over the zero-shot-no-guidelines baseline by up to 9%. There are minimal performance gains in the two-shot-with-guidelines case over the zero-shot-with-guidelines and two-shot-no-guidelines cases. However, inclusion of both does reduce the frequency of errors which is key for our automated workflow. Our pipeline outperforms other modelling pipeline in Section 4.4 (though NuExtract may have potential) and, perhaps bar spaCy via its rules, is more interpretable because of the guideline creation.

**Refining the LAAJ prompt.** We tried a few different ideas for the LAAJ prompt but ended up settling on a simpler shorter version preceded by if-else statements for edge cases. Longer prompts included rules and/or few-shot samples per type of pair (exact, same concept, similar enough, different) but these only seemed to confuse the evaluator model. We were unable to convert phrases to "canonical forms" (beyond making them all lower case) for easier comparison due to the variability in phrasing. However, edge cases should still be included in the `laaj_test_cases.py` file so annotators are explicitly aware of where their model works and where it does not.

The workflow to tweaking the LAAJ prompt is as follows: (1) examine *string* entities such as *chronic change* and *final diagnosis*, comparing human annotations, LLM annotations, and their predictions, (2) add entity pairs which you think the LLM has incorrectly predicted as True or False to `laaj_test_cases.py`, (3) refine the prompt to cover as many of these cases as possible. Some other important cases to include are ranges (adjectives e.g. "mild to moderate" or numerical e.g. "5-10%"), common misspellings, domain-specific phrases for normality (e.g. "morphologically normal", "unremarkable", "no abnormalities"), domain-specific synonyms (e.g. ("uromodulin", "Tamm-Horsfall protein"), ("nuclear loss", "pyknosis")), and cases where severity is ambiguous (e.g. "mild chronic change" vs "mild to moderate chronic change"). We reiterate what we said before: **the aim is not to achieve perfect performance here, it is to know what you want your LLM evaluator to output and what it actually outputs so you know what it is confident and unconfident on**; the unconfident cases can be given to clinicians. On a related practical point, it is useful to keep your saved annotations from previous runs in case you tweak the LAAJ prompt since all you need to update is the evaluation, not the annotation. An alternative classical approach worth exploring, which we did not due to time constraints, could be using BERT embedding scores to assess similarity between responses.

**Refining the entity schema and guidelines.** We have developed an efficient annotation workflow that produces structured outputs for medical reports. For the renal biopsy dataset, the next step is creating a more granular entity schema covering glomerular abnormalities, cell infiltrate composition, vascular dysfunction, and standardised diagnostic terminology. We ran out of time to do this, but would recommend separating rejection grades from diseases rather than combining them into one *final diagnosis* string. While our prompts align with clinical guidelines, they require explicit phrasing to minimise LLM ambiguity. An easy categorisation to help refine the schema is to break up groups of entities into cases consistent with healthy patients, standard diseased patients, complex diseased patients, and edge cases. Taking the sclerosed glomeruli as our example, normal patients will have no globally or segmentally sclerosed glomeruli, standard diseased patients will have globally sclerosed glomeruli, complex diseased patients will have globally sclerosed glomeruli, non-sclerotic abnormalities, and potentially segmentally sclerosed glomeruli, and edge cases include ambiguities, e.g., no reference to glomeruli or unknown numbers of sclerosed glomeruli. **Your**

**aim should be to align your annotations with the LLM's annotations**.

**Practical considerations.** The workflow was designed with a minimal number of required clinical touchpoints. Over the course of this work, there were four meetings with a clinician over three months. For these hour-long meetings, we recommend preparing questions on optimal entity guidelines and observed annotation errors. The speed of the models is not particularly important to clinicians; it is important to data scientists for their optimal iteration workflows. There is also the challenge of working within your model's context window - 8K windows require concise entity guidelines and a limited number of few-shots. There is also the open research question of how many annotations are needed to refine the guideline schema before performance deteriorates - we leave this for future work.

## 5.2 Limitations and Future Work

**Performance improvement.** Several approaches could improve the pipeline's performance beyond tweaking the entity guidelines and simply adding more few-shot examples. Combining spaCy outputs with the *final diagnosis* entity from BioBERT-SQuAD would achieve 78.3% accuracy across 100 reports, outperforming all zero-shot LLM models without guidelines. A promising direction is combining multiple models: using spaCy for binary entities (cortex, medulla, transplant), LLMs for complex string entities (chronic changes, diagnoses), and both in a disagreement modelling framework for numerical entities that require co-referencing and well-established default values. The flexibility of LLM pipelines remains their key advantage when prompts and few-shots are properly constrained to the medical domain. Other enhancements could include optimising inference by including chain-of-thought for interpretability and debugging purposes and smart filtering of ensemble outputs for improved performance.

**False Confidence and Prompt Sensitivity.** These are two key limitations which are relevant to deployment. Disagreement modelling helps us compare models and improves the process of error analysis and robust prompt creation. However, it does not solve the problem of false predictions which is more acute if both models agree on their incorrect predictions. We hope to have de-risked this issue by verifying our models work against a smaller sample of human-annotated reports and using a clinician to verify poor alignment between the models, but neither of these are a guarantee on perfect annotation. A more involved disagreement modelling approach could use two or more models with complementary strengths (as described in the previous paragraph) to produce more reliable annotations. The other limitation is that we have not conducted a comprehensive prompt sensitivity analysis. Small changes in guidelines and prompt structure should not alter the results significantly, and this is particularly important for the aim of human-in-the-loop annotation where changes are made quickly and, perhaps, suboptimally. A potential solution is to create a standardised format for entity guidelines (maybe corresponding to our rules) and to define key phrases that can be used across multiple entities in a schema (e.g. if something is missing, specify it should be "null" rather than "undefined", "not present", etc.) so that prompts are robust.

**Tighter integration between data scientist, clinician, and LLM.** Another important area is improving the *asynchronous* data scientist-clinician annotation loop through better tooling and communication channels. We have three clinician touchpoints to optimise (how the data scientist uses) clinician time but this is a minimum requirement that can be increased for specific workflows. The Streamlit app is a good start but it is brittle and software engineering work is required to turn it into something more stable. This could take the form of a `.exe` file that the clinician could download from the GOSH Software Centre or a webapp that exists on GOSH's Aridhia server so it could be used by anyone with GOSH access. The app

could include an open communication channel for doctors/annotators to regularly flag UX issues - something we have briefly gotten feedback on by showing short video demos of the Streamlit app and sending wireframes to potential annotators. Future work could run studies to evaluate AI-integrated annotation workflows taking inspiration from human-in-the-loop workflows for automated clinical coding that evaluated the impact of using AI for different levels of human coding experience [31].

**Building an annotation agent.** We define an agent as a self-updating program based around prompts. There are several data scientist tasks in Figure 3 that have the potential for further automation, i.e., they can be described as a prompt replicating human processes. We identify the following processes that can be "promptified": (1) getting a high-level understanding of the biological area, (2) entity selection, (3) report header identification, (4) guideline creation, (5) few-shot creation (similarity via embeddings, report complexity via guideline "hits"), and (6), qualitative error analysis. Future work could combine these steps into an agent that updates each prompt based on output annotation performance. Long context models would be required to include all corpus information in the prompt, and the main challenge here is ensuring sufficient compute. There were also approximately 350 reports that we filtered out during EDA because they were a different format to most reports. More flexible preprocessing workflows inspired by LLM-based document processors like DocETL [38] could be helpful here.

**Generalising to other areas of biology.** Extending our approach to other biomedical areas (that GOSH has reports for), such as foetal post-mortem analyses, non-specific tumour classification and genomics discovery, presents additional challenges and opportunities. Most of these fall under the umbrella of being good ways to standardise report information and create high-quality signals for multi-modal models (e.g., VLMs for histopathology reports, genomic-language models for genomics reports), some of which could be used for diagnostics and biomedical discovery. However, how the outputs fit into the care pathway vary considerably. Renal biopsies are a well-understood area of biology where there is often a clear next step of treatment, while foetal post-mortems have multiple "next" steps. They have the immediate impact of helping parents find closure after miscarriages, then, in the medium-term, provide information for future pregnancies, and in a broader sense, reduce the stigma around discussing these difficult topics [39, 40]. Genomics, in many ways, is a rapidly evolving science and while next stages of treatment are not always clear, ongoing development of gene therapies could alter these care pathways significantly, as they did recently in the case of UK's recent approval of Trikafta which is a cure for 90% of cystic fibrosis (a monogenic disease) patients. Many of these challenges porting to new domains could be tackled by using the additional clinical and disease information in the generation process, either by using longer context models with the prior knowledge included in the prompt (i.e., longer guidelines), or smart retrieval-augmented generation frameworks such as contextual retrieval [41].

Getting the scope right for each of these cases is important. The overarching scope for the renal biopsy dataset was separation between native and transplant biopsies (encoded as the boolean *transplant* entity) which have some similar, but also some unique, symptoms. Similar clinically-relevant biopsy subgroups should be identified a priori by talking to clinicians. For example, maybe genomics alone is too broad but genomics for monogenic diseases might be a more standardised subgroup and pathway. Additionally, there are metadata confounders that could affect performance here, such as temporal information. For renal biopsy reports, some baseline level of glomerulosclerosis is expected in obese patients - this fact is particularly important if projects progress beyond data organisation into discovery science. We do not consider these confounders but future work could integrate them into the entity guide-

lines schema.

### 5.3 Broader Considerations

**Extending to other trusts.** GOSH is a internationally-known paediatric hospital in London and so gets more funding than most trusts. Since we only have CPUs[3], it is likely that other trusts will only have CPU infrastructure too. Fortunately, since our pipeline can run in reasonable time with a 16GB RAM laptop, other trusts with enough budget for similarly-specced hardware should be able to use our code, provided they are able to access their trust's medical reports. GOSH is also unique in that it is a tertiary care (which essentially means "specialised care") hospital which receives paediatric patients from all around the UK (maybe even Europe) and so has extremely varied presentations contained in medical reports. Other trusts with adult patients localised to that area are more likely to have similar environmental factors affecting them and probably more similar presentations. In other words, our pipeline works on the hardest possible version (paediatric care with highly variable clinical pathways) of a simple test problem (renal biopsy). It is likely that our pipeline at least works for adult renal biopsy reports, if not for more complex biology, though more testing is required.

**Standardising reports within a trust and between trusts.** Although we have chosen to remove information of clinician authorship for simplicity, this information could inform quality improvement by understanding and minimising differences in how clinicians write reports. In doing so, though, this targeted "training" for clinicians would raise concerns about clinician privacy and fair assessment. Testing model generalisation across institutions is crucial, as is understanding how symptoms, guidelines and documentation practices vary between hospitals. Early integration of language models into clinical workflows would ease future adoption of vision-language models, while ensuring standardisation aligns with each trust's specific needs. This approach would support the broader goal of training more pathologists while maintaining reporting quality.

**Ethical considerations.** Paediatric contexts have specific issues around children's medical consent and preferences, as well as data privacy. A recent study looking into the attitudes of fracture patients at GOSH [42] found that the majority of children wanted AI practitioners to prioritise model accuracy over speed and preferred clinician-in-the-loop (CITL) modelling workflows. Regarding the accuracy-speed tradeoff, if this pipeline were to be deployed in production settings, the speed requirements used here of two days may be relaxed to improve performance. Regarding CITL workflows, we have sought to include clinicians in a practical way, hopefully achieving this aim. Privacy concerns are also amplified in paediatric medicine because there are lower disease incidence rates compared to adult populations. This creates challenges for training models since there is an increased risk of data re-identification, particularly in rare diseases where unique case characteristics could compromise anonymity. In our codebase, we do not publish any data other than our entity guidelines and synthetic few shot examples, both of which contain no unique patient information.

## 6 Conclusion

We have presented an iterative approach to annotating and extracting structured information from histopathology reports using LLMs, incorporating expert knowledge through three key touchpoints for refining annotation guidelines and few-shot examples. We achieve strong

[3] Although during this internship, GOSH did secure a GPU grant for computer vision work so perhaps this is changing!

annotation performance on a sample of histopathology reports for renal biopsies that outperforms other zero-shot information extraction approaches like spaCy, BERT QA models, GLiNER, and NuExtract. Our workflow includes a section on disagreement modelling which can be used for automated annotation and prompt robustness tests. A key practical advantage of our approach is its accessibility - the entire workflow can run on a standard 16GB RAM laptop, making it feasible for research purposes across NHS trusts.

# 7 Acknowledgements

Thank you to Jenny Chim, Scarlett Kynoch, the rest of the NHSE Data Science team, Daniel Key, the rest of the GOSH DRIVE team, Joram Posma, and Kristen Severson for helpful discussions around lightweight training of models, consistent output generation for evaluation, working on GOSH infrastructure, and optimising prompts.

# References

[1] Bean, D. M., Kraljevic, Z., Shek, A., Teo, J. & Dobson, R. J. Hospital-wide natural language processing summarising the health data of 1 million patients. *PLOS digital health* **2**, e0000218 (2023).

[2] Eyre, H. *et al.* Launching into clinical space with medspacy: a new clinical text processing toolkit in python. In *AMIA Annual Symposium Proceedings*, vol. 2021, 438 (2022).

[3] Kraljevic, Z. *et al.* Multi-domain clinical natural language processing with medcat: the medical concept annotation toolkit. *Artificial intelligence in medicine* **117**, 102083 (2021).

[4] Honnibal, M. Against llm maximalism. https://explosion.ai/blog/against-llm-maximalism (2023). Accessed: 26-11-2024.

[5] Jabarulla, M. Y., Oeltze-Jafra, S., Beerbaum, P. & Uden, T. Meddoc-bot: A chat tool for comparative analysis of large language models in the context of the pediatric hypertension guideline. *arXiv preprint arXiv:2405.03359* (2024).

[6] Barile, J. *et al.* Diagnostic accuracy of a large language model in pediatric case studies. *JAMA pediatrics* **178**, 313–315 (2024).

[7] Andrew, J. J., Vincent, M., Burgun, A. & Garcelon, N. Evaluating llms for temporal entity extraction from pediatric clinical text in rare diseases context. In *Proceedings of the First Workshop on Patient-Oriented Language Processing (CL4Health)@ LREC-COLING 2024*, 145–152 (2024).

[8] Yang, D. *et al.* Pediatricsgpt: Large language models as chinese medical assistants for pediatric applications. *arXiv preprint arXiv:2405.19266* (2024).

[9] Neumann, M., King, D., Beltagy, I. & Ammar, W. Scispacy: fast and robust models for biomedical natural language processing. *arXiv preprint arXiv:1902.07669* (2019).

[10] Lee, J. *et al.* Biobert: a pre-trained biomedical language representation model for biomedical text mining. *Bioinformatics* **36**, 1234–1240 (2020).

[11] Alsentzer, E. *et al.* Publicly available clinical bert embeddings. *arXiv preprint arXiv:1904.03323* (2019).

[12] Zaratiana, U., Tomeh, N., Holat, P. & Charnois, T. Gliner: Generalist model for named entity recognition using bidirectional transformer. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies* (*Volume 1: Long Papers*), 5364–5376 (2024).

[13] Wang, S. *et al.* Gpt-ner: Named entity recognition via large language models. *arXiv preprint arXiv:2304.10428* (2023).

[14] Ashok, D. & Lipton, Z. C. Promptner: Prompting for named entity recognition. *arXiv preprint arXiv:2305.15444* (2023).

[15] Bogdanov, S., Constantin, A., Bernard, T., Crabbé, B. & Bernard, E. Nuner: Entity recognition encoder pre-training via llm-annotated data. *arXiv preprint arXiv:2402.15343* (2024).

[16] NuMind. Nuextract: A foundation model for structured extraction. https://numind.com/nuextract (2024). Accessed: 05-12-2024.

[17] Hicks, M. T., Humphries, J. & Slater, J. Chatgpt is bullshit. *Ethics and Information Technology* **26**, 38 (2024).

[18] Zeng, K. G. *et al.* Improving information extraction from pathology reports using named entity recognition. *Research Square* (2023).

[19] Santos, T. *et al.* Pathologybert-pre-trained vs. a new transformer language model for pathology domain. In *AMIA annual symposium proceedings*, vol. 2022, 962 (American Medical Informatics Association, 2022).

[20] Peng, Y., Chen, Q. & Lu, Z. An empirical study of multi-task learning on bert for biomedical text mining. In *Proceedings of the 19th SIGBioMed Workshop on Biomedical Language Processing*, 205–214 (2020).

[21] Mu, Y. *et al.* A bert model generates diagnostically relevant semantic embeddings from pathology synopses with active learning. *Communications medicine* **1**, 11 (2021).

[22] Lu, Y. *et al.* Assessing the impact of pretraining domain relevance on large language models across various pathology reporting tasks. *medRxiv* 2023–09 (2023).

[23] Bumgardner, V., Mullen, A., Armstrong, S., Hickey, C. & Talbert, J. Local large language models for complex structured medical tasks. *arXiv preprint arXiv:2308.01727* (2023).

[24] Tan, Z. *et al.* Large language models for data annotation and synthesis: A survey. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 930–957 (2024).

[25] Xiao, R. *et al.* Freeal: Towards human-free active learning in the era of large language models. *arXiv preprint arXiv:2311.15614* (2023).

[26] Munnangi, M. *et al.* On-the-fly definition augmentation of llms for biomedical ner. *arXiv preprint arXiv:2404.00152* (2024).

[27] Naraki, Y., Yamaki, R., Ikeda, Y., Horie, T. & Naganuma, H. Augmenting ner datasets with llms: Towards automated and refined annotation. *arXiv preprint arXiv:2404.01334* (2024).

[28] Jeong, S., Baek, J., Cho, S., Hwang, S. J. & Park, J. C. Test-time self-adaptive small language models for question answering. *arXiv preprint arXiv:2310.13307* (2023).

[29] Warmerdam, V. Large disagreement models. https://koaning.io/posts/large-disagreement-models/ (2023). Accessed: 18-12-2024.

[30] Montani, I. A practical guide to human-in-the-loop distillation. https://explosion.ai/blog/human-in-the-loop-distillation (2024). Accessed: 18-12-2024.

[31] Gao, Y. *et al.* Optimising the paradigms of human ai collaborative clinical coding. *npj Digital Medicine* **7**, 368 (2024).

[32] Pope, R. *et al.* Real-world learnings for digital health industry–nhs collaboration: life sciences vision in action. *Future Healthcare Journal* **11**, 100168 (2024).

[33] Loupy, A., Mengel, M. & Haas, M. Thirty years of the international banff classification for allograft pathology: the past, present, and future of kidney transplant diagnostics. *Kidney international* **101**, 678–691 (2022).

[34] Miranda, L. Some thoughts on the annotation process. https://ljvmiranda921.github.io/notebook/2023/07/03/devlog-calamancy/ (2023). Accessed: 26-11-2024.

[35] Miranda, L. J. Developing a named entity recognition dataset for tagalog. In *Proceedings of the First Workshop in South East Asian Language Processing*, 13–20 (2023).

[36] Rajpurkar, P., Jia, R. & Liang, P. Know what you don't know: Unanswerable questions for squad. *arXiv preprint arXiv:1806.03822* (2018).

[37] Liu, Y. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692* **364** (2019).

[38] Shankar, S., Parameswaran, A. G. & Wu, E. Docetl: Agentic query rewriting and evaluation for complex document processing. *arXiv preprint arXiv:2410.12189* (2024).

[39] Hospital, G. O. S. Helping parents find answers after miscarriage. https://www.gosh.nhs.uk/news/helping-parents-find-answers-after-miscarriage/ (2024). Accessed: 26-11-2024.

[40] Shelmerdine, S. & Davendralingam, N. Radiology's role in humanising mortality. *BMJ* **388** (2025).

[41] Anthropic. Introducing contextual retrieval. https://www.anthropic.com/news/contextual-retrieval (2024). Accessed: 05-12-2024.

[42] Lee, L. *et al.* "how i would like ai used for my imaging": children and young persons' perspectives. *European Radiology* 1–14 (2024).